



中国科学技术大学
University of Science and Technology of China

计算机系统概论

Introduction to Computing Systems
(CS1002A.03)

Chapter 8-2

Recursion, Queue and Character Strings

陈俊仕

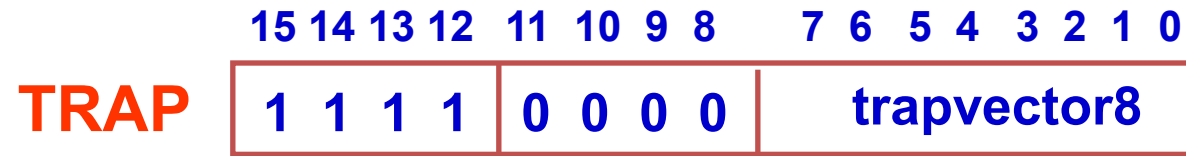
cjuns@ustc.edu.cn

2025 Fall

计算机科学与技术学院

School of Computer Science and Technology

TRAP Instruction



- **Trap vector**
 - Identifies which system call to invoke
 - Serves as index into table of service routine addresses
 - LC-3: table stored in memory at 0x0000 – 0x00FF
 - 8-bit trap vector zero-extended to form 16-bit address
 - Enters privileged mode (PSR[15]<-0)
- **Where to go**
 - Lookup starting address from table; place in PC
- **Enabling return**
 - Save PC after TRAP and PSR to system stack
 - Saved_USP <- R6, Saved_SSP <- R6

- **RTI – Return from Trap or Interrupt instruction**
 - How to return
 - Pop the top two values on the system stack into the PC and PSR.
 - Place address in R7 in PC, return the execution to the last calling point.
 - $R6 \leftarrow \text{Saved_USP}(\text{PSR}[15]==1) \text{ or } \text{Saved_SSP}(\text{PSR}[15]==0)$

RTI

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

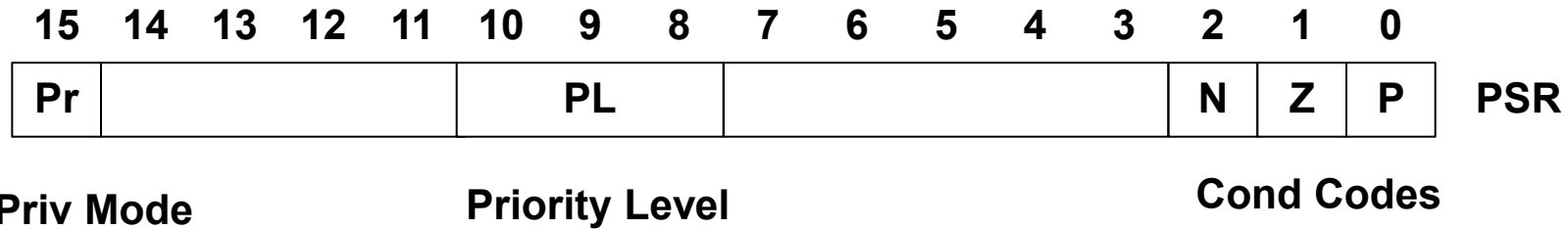
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The Processor State Register(PSR)

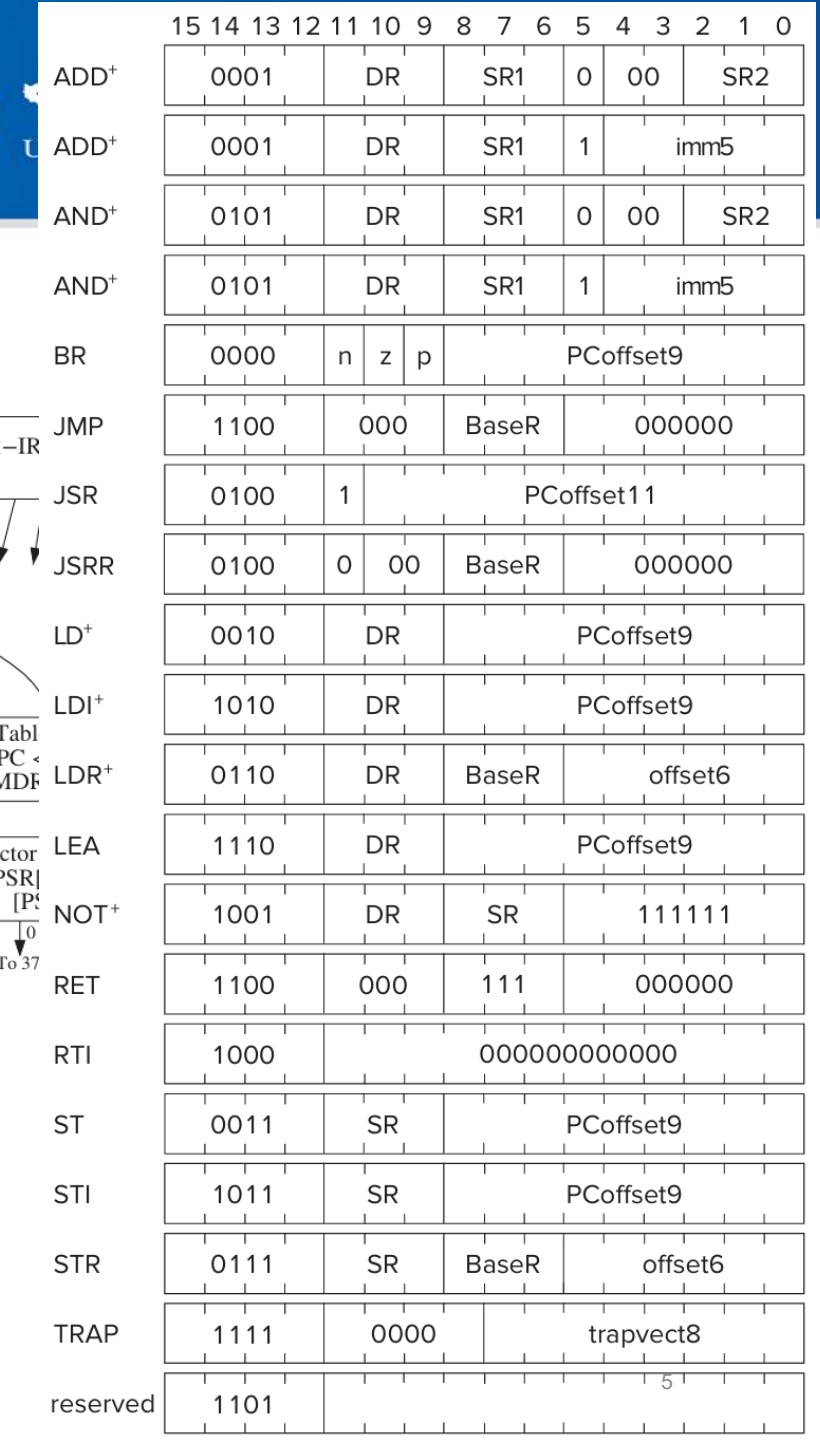


中国科学技术大学
University of Science and Technology of China

The Processor Status Register

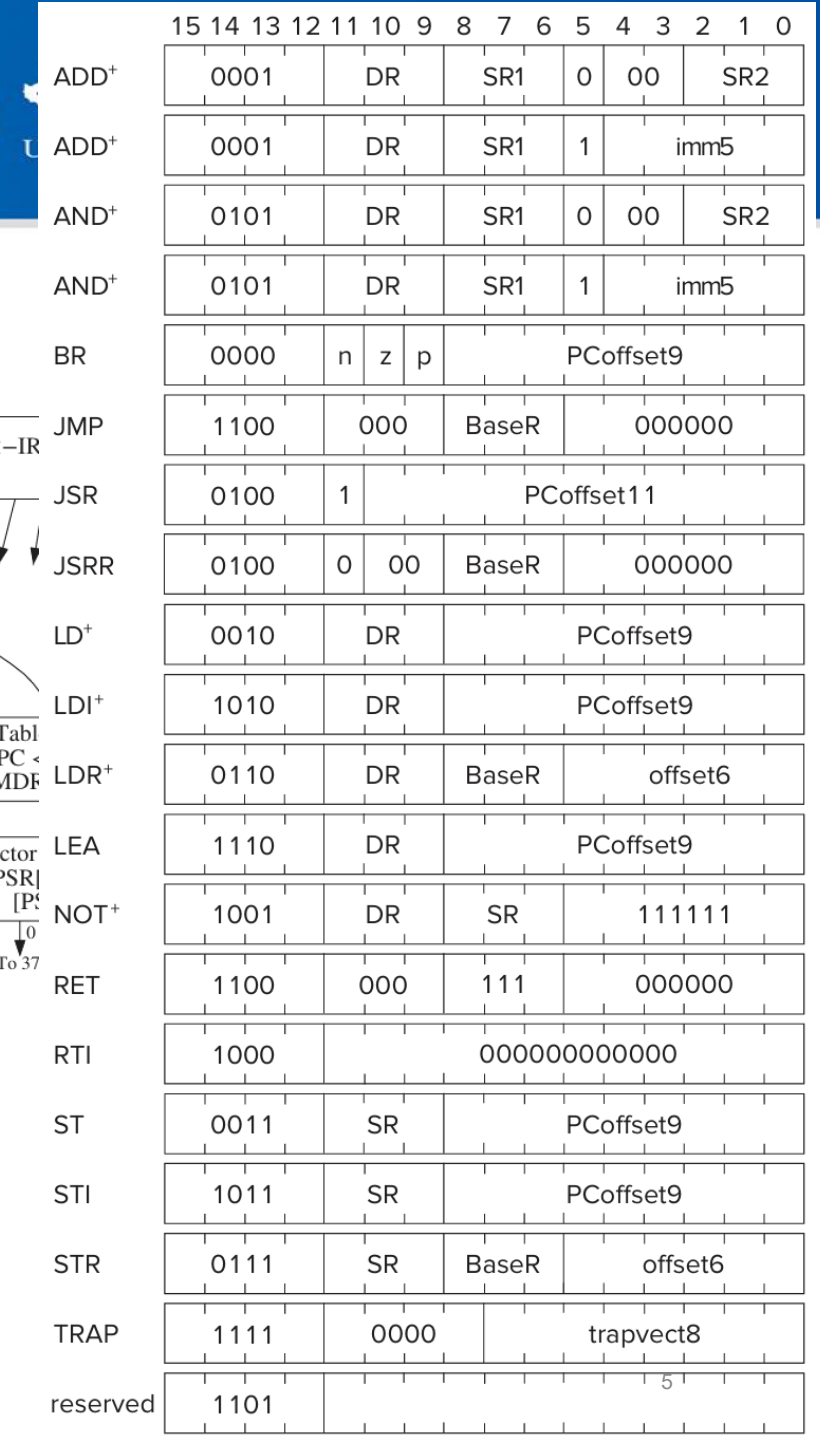


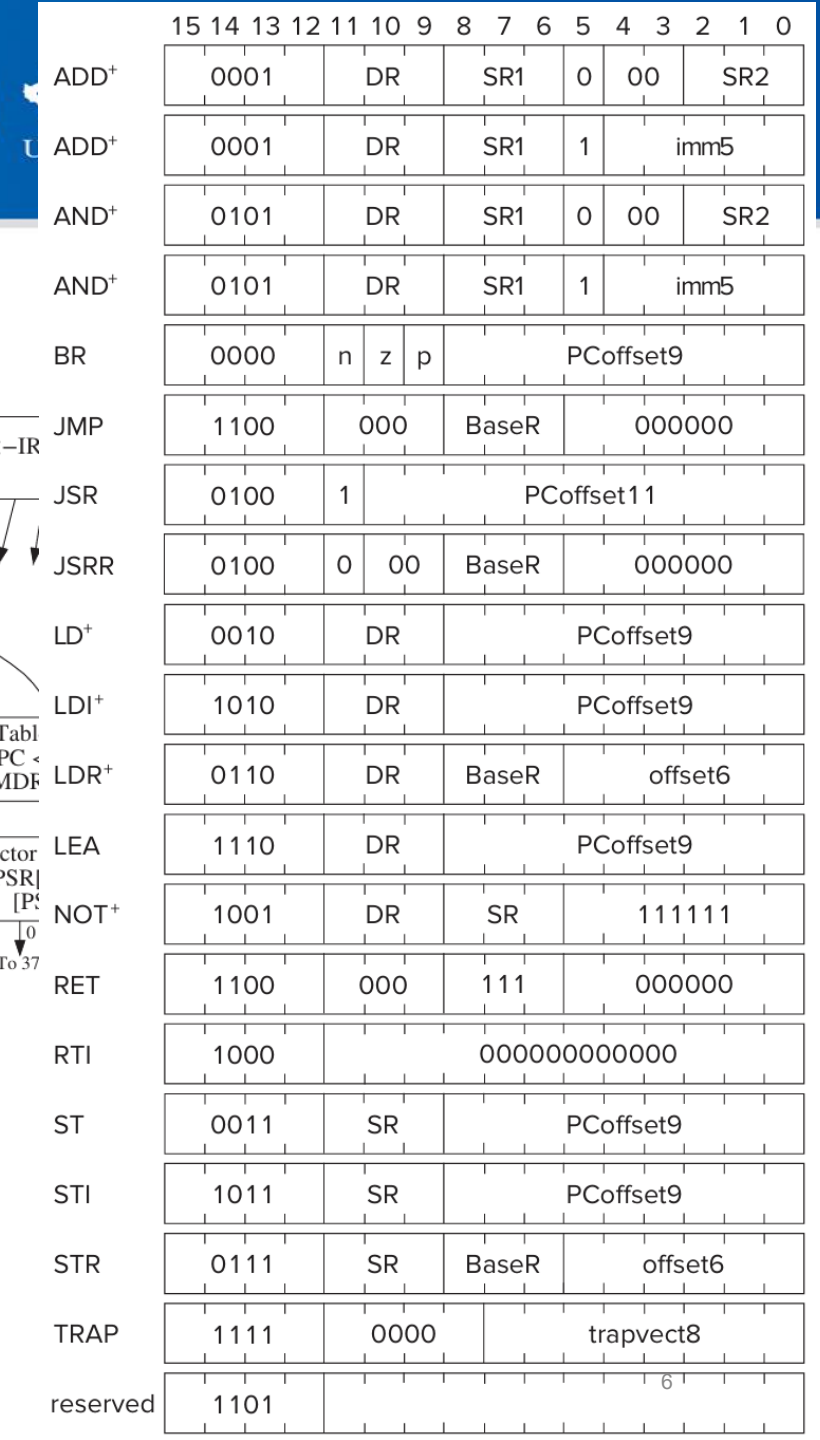
- **PSR[15]**
 - 0, privileged (supervisor) mode
 - 1, unprivileged(user)mode
- The Priority Level of a program being executed
 - PL[10:8]
 - 0(最低)~7(最高)



	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001			DR			SR1			0	00		SR2			
ADD ⁺	0001			DR			SR1			1	imm5					
AND ⁺	0101			DR			SR1			0	00		SR2			
AND ⁺	0101			DR			SR1			1	imm5					
BR	0000			n	z	p	PCOffset9									
JMP	1100			000			BaseR			000000						
JSR	0100			1	PCOffset11											
JSRR	0100			0	00		BaseR			000000						
LD ⁺	0010			DR			PCOffset9									
LDI ⁺	1010			DR			PCOffset9									
LDR ⁺	0110			DR			BaseR			offset6						
LEA	1110			DR			PCOffset9									
NOT ⁺	1001			DR			SR			111111						
RET	1100			000			111			000000						
RTI	1000			000000000000												
ST	0011			SR			PCOffset9									
STI	1011			SR			PCOffset9									
STR	0111			SR			BaseR			offset6						
TRAP	1111			0000			trapvect8									
reserved	1101			5												

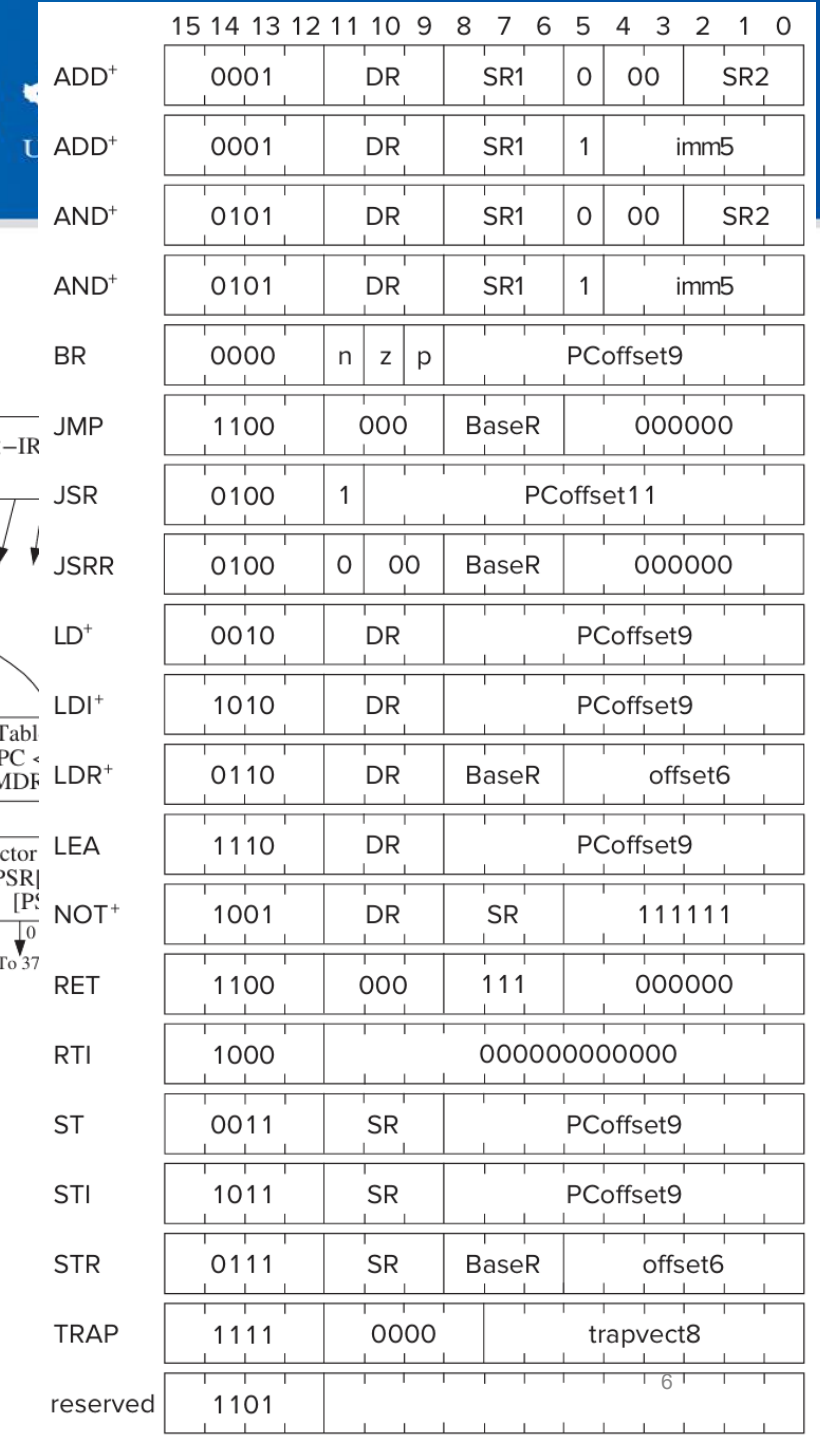
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001			DR			SR1			0	00		SR2			
ADD ⁺	0001			DR			SR1			1	imm5					
AND ⁺	0101			DR			SR1			0	00		SR2			
AND ⁺	0101			DR			SR1			1	imm5					
BR	0000			n	z	p	PCOffset9									
JMP	1100			000			BaseR			000000						
JSR	0100			1	PCOffset11											
JSRR	0100			0	00		BaseR			000000						
LD ⁺	0010			DR			PCOffset9									
LDI ⁺	1010			DR			PCOffset9									
LDR ⁺	0110			DR			BaseR			offset6						
LEA	1110			DR			PCOffset9									
NOT ⁺	1001			DR			SR			111111						
RET	1100			000			111			000000						
RTI	1000			000000000000												
ST	0011			SR			PCOffset9									
STI	1011			SR			PCOffset9									
STR	0111			SR			BaseR			offset6						
TRAP	1111			0000			trapvect8									
reserved	1101			5												





	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001			DR			SR1			0		00		SR2		
ADD ⁺	0001			DR			SR1			1		imm5				
AND ⁺	0101			DR			SR1			0		00		SR2		
AND ⁺	0101			DR			SR1			1		imm5				
BR	0000			n	z	p	PCOffset9									
JMP	1100			000			BaseR			000000						
JSR	0100			1			PCOffset11									
JSRR	0100			0	00		BaseR			000000						
LD ⁺	0010			DR			PCOffset9									
LDI ⁺	1010			DR			PCOffset9									
LDR ⁺	0110			DR			BaseR			offset6						
LEA	1110			DR			PCOffset9									
NOT ⁺	1001			DR			SR			111111						
RET	1100			000			111			000000						
RTI	1000			000000000000												
ST	0011			SR			PCOffset9									
STI	1011			SR			PCOffset9									
STR	0111			SR			BaseR			offset6						
TRAP	1111			0000			trapvect8									
reserved	1101			6												

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001			DR			SR1			0		00		SR2		
ADD ⁺	0001			DR			SR1			1		imm5				
AND ⁺	0101			DR			SR1			0		00		SR2		
AND ⁺	0101			DR			SR1			1		imm5				
BR	0000			n	z	p	PCOffset9									
JMP	1100			000			BaseR			000000						
JSR	0100			1			PCOffset11									
JSRR	0100			0	00		BaseR			000000						
LD ⁺	0010			DR			PCOffset9									
LDI ⁺	1010			DR			PCOffset9									
LDR ⁺	0110			DR			BaseR			offset6						
LEA	1110			DR			PCOffset9									
NOT ⁺	1001			DR			SR			111111						
RET	1100			000			111			000000						
RTI	1000			000000000000												
ST	0011			SR			PCOffset9									
STI	1011			SR			PCOffset9									
STR	0111			SR			BaseR			offset6						
TRAP	1111			0000			trapvect8									
reserved	1101			6												





1 Recursion

2 The Queue

3 Character Strings

1. Recursion

- Recursion is a mechanism for expressing a function *in terms of itself*.
- When used appropriately, the expressive power of recursion is going to save us a lot of headaches
- Otherwise, it results in **longer execution time** and **wasted energy** (High-Carbon)

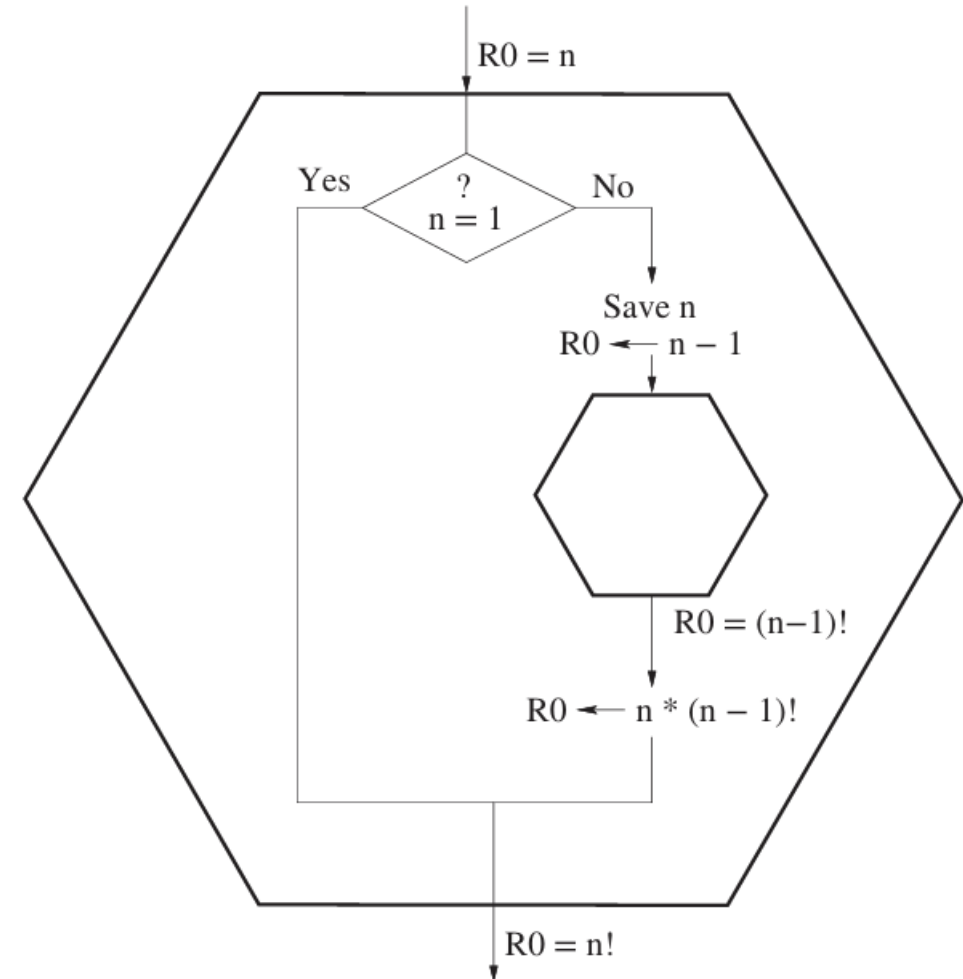
Bad Example: Factorial

■ To compute $n! = n * (n-1)!$

- assume the LC-3 has a **MUL** instruction

■ C: *int fact(int n)*

```
FACT  ADD R1,R0,#-1
      BRz DONE
; call FACT(n-1)
      ADD R1,R0,#0
      ADD R0,R1,#-1
B     JSR FACT
; n * fact(n-1)
      MUL R0,R0,R1
; return
DONE  RET
```



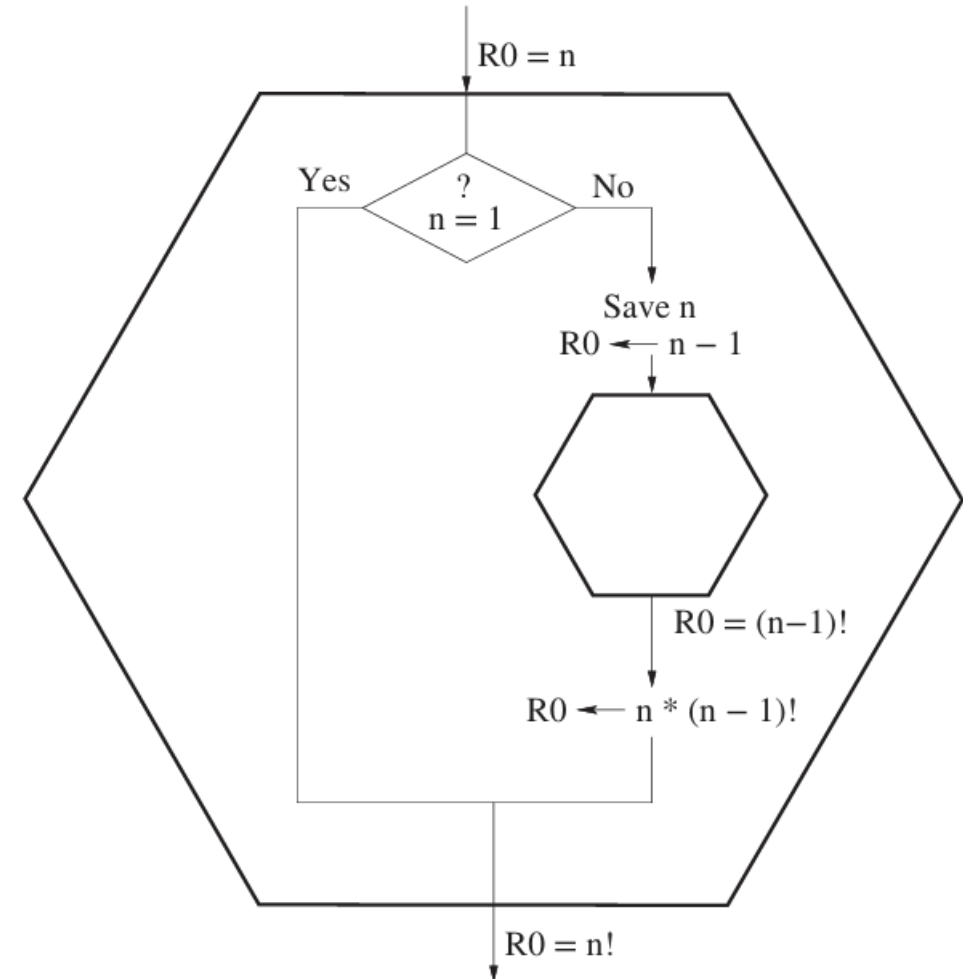
Bad Example: Factorial

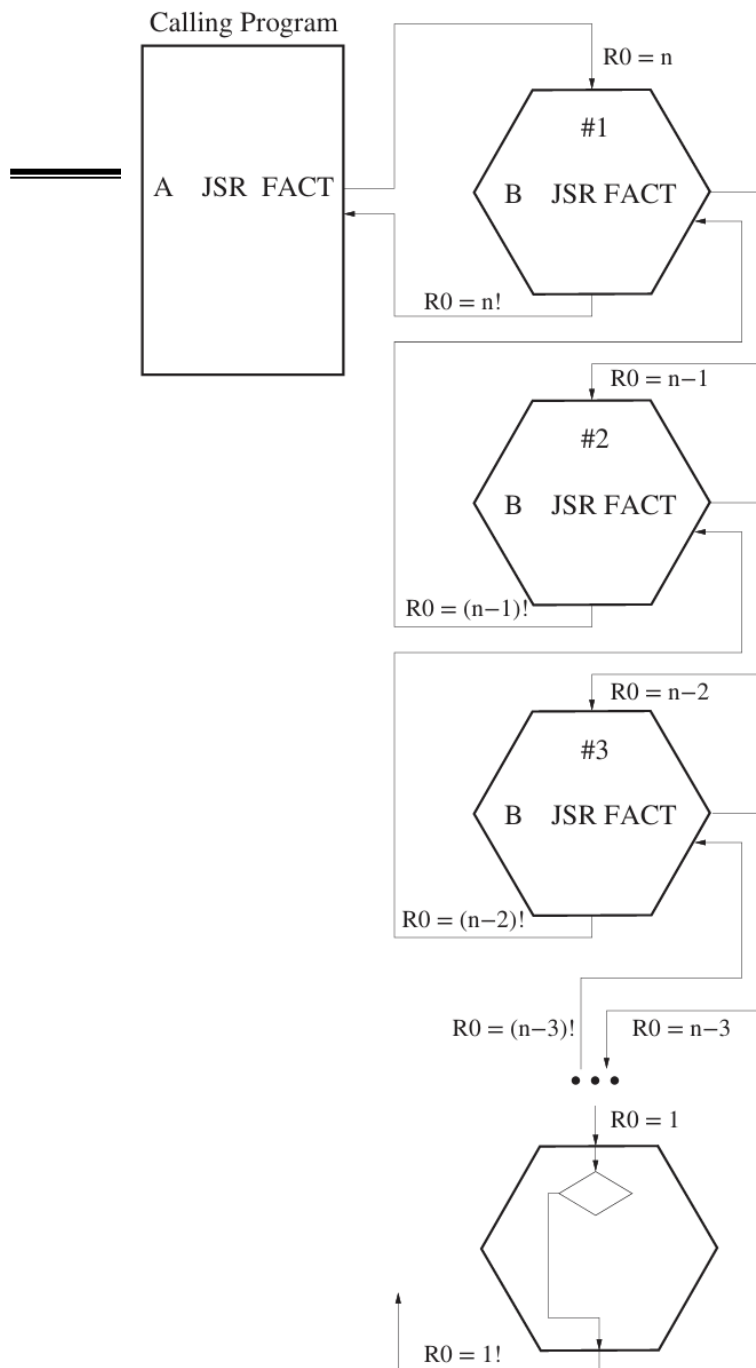
■ To compute $n! = n * (n-1)!$

- assume the LC-3 has a **MUL** instruction

■ C: *int fact(int n)*

```
FACT  ST  R1, Save1
      ADD R1, R0, #-1
      BRz DONE
; call FACT(n-1)
      ADD R1, R0, #0
      ADD R0, R1, #-1
B     JSR FACT
; n * fact(n-1)
      MUL R0, R0, R1
; return
DONE  LD  R1, Save1
      RET
Save1 .BLKW 1
```





Bad Example: Factorial

In Calling Program: **A JSR FACT:** **A+1 → R7**

In #1: **B JSR FACT:** **B+1 → R7**

R7=A+1 is wiped out by B+1, can not return.

```

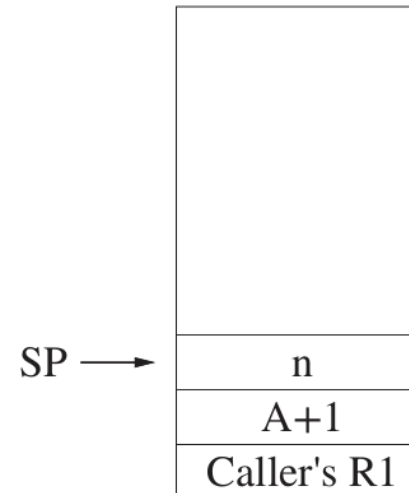
FACT  ST  R1,Save1
      ADD R1,R0,#-1
      BRz DONE
; call FACT(n-1)
      ADD R1,R0,#0
      ADD R0,R1,#-1
B   JSR FACT
; n * fact(n-1)
      MUL R0,R0,R1
; return
DONE  LD  R1,Save1
      RET
Save1 .BLKW 1
  
```


Recursive subroutine FACT

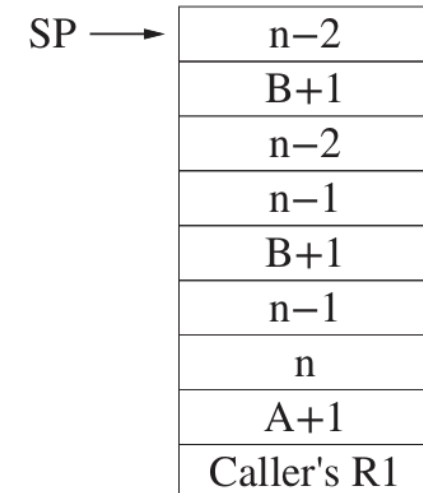
```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller' s R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller' s R1 back into R1
■          ADD R6,R6,#1
■          RET
```

Solution: save R7 to stack

a. Contents of stack when
JSR FACT executes in #1



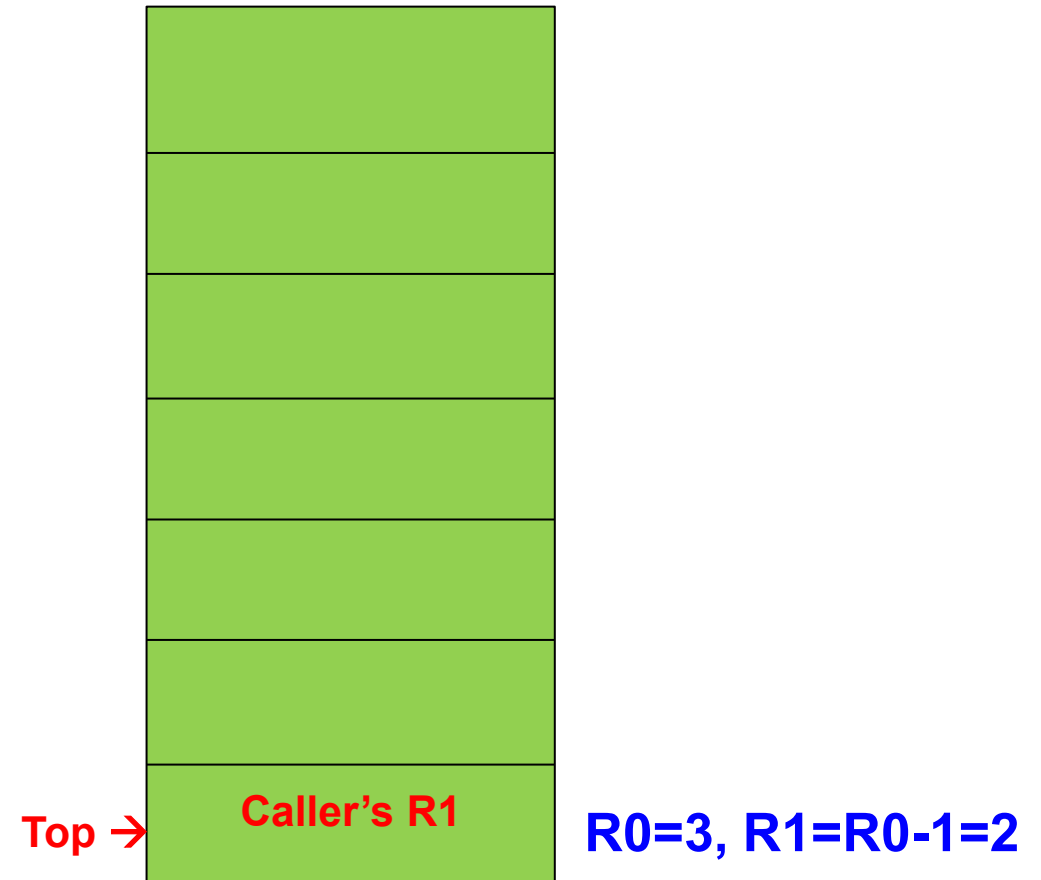
a. Contents of stack when
JSR FACT executes in #3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■          ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■          ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■          ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■          ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

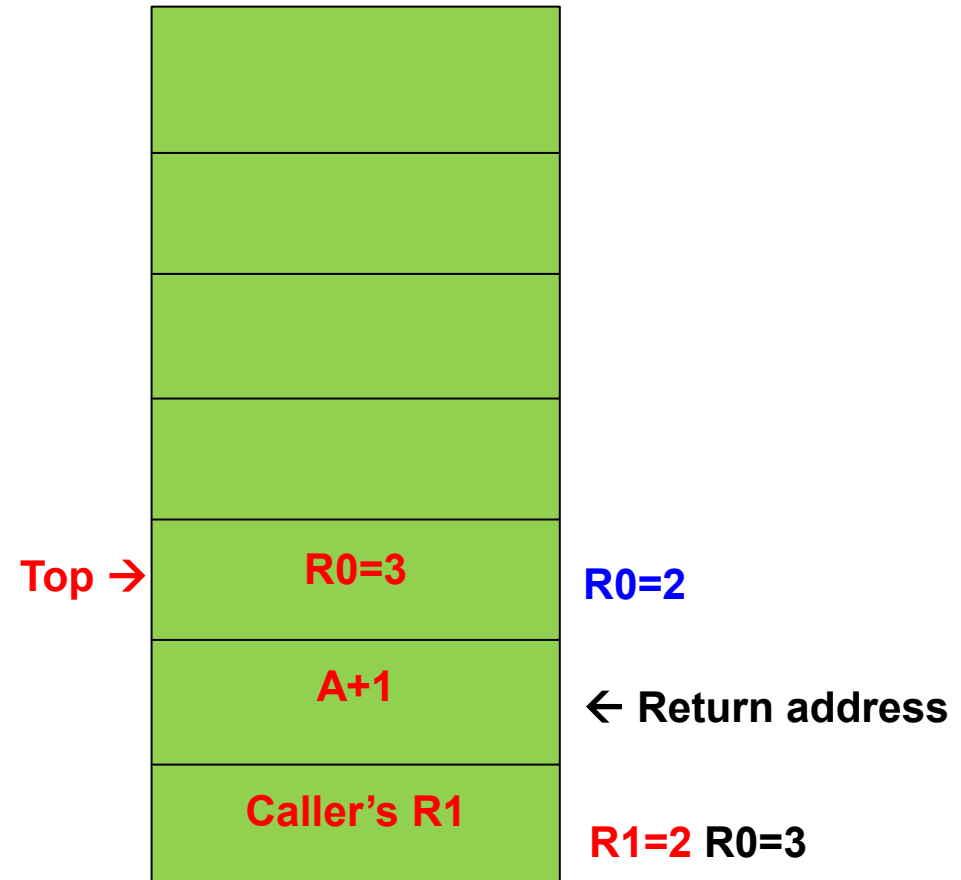
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

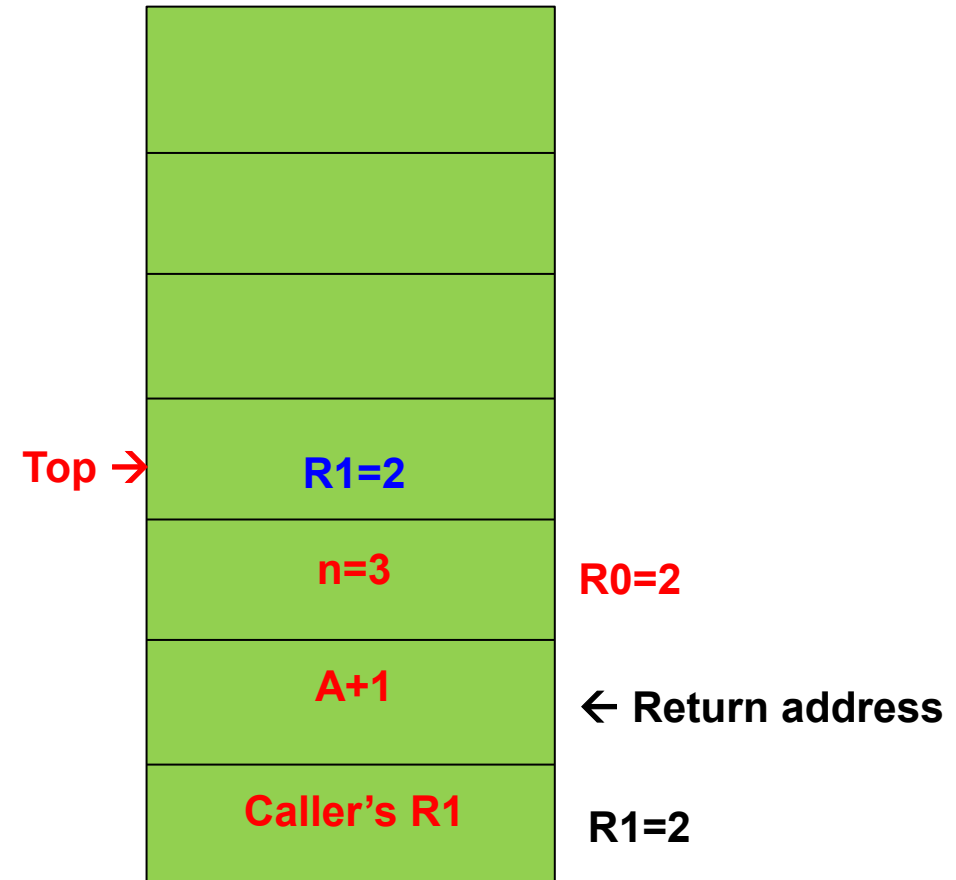
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■          ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■          ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■          ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT    ;B+1→R7
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■          ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

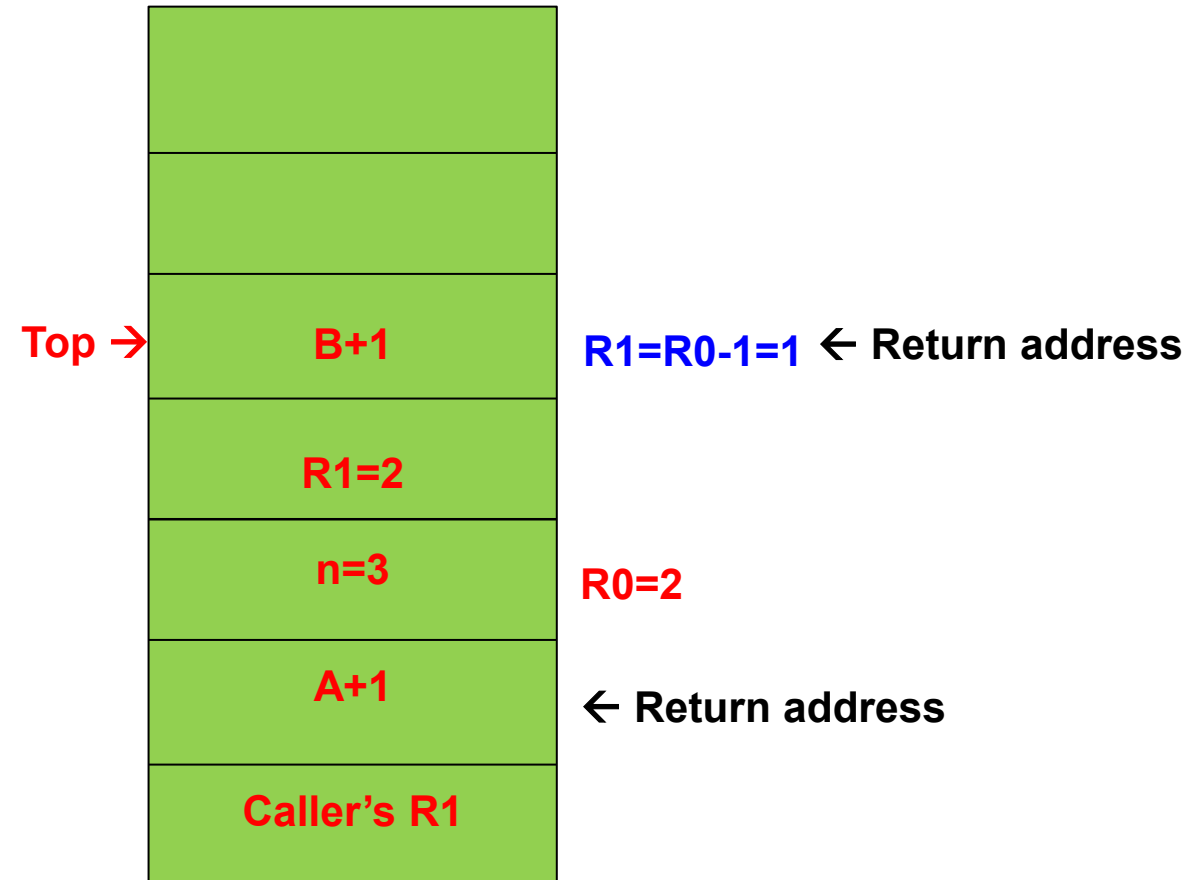
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■          ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■          ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■          ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■          ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

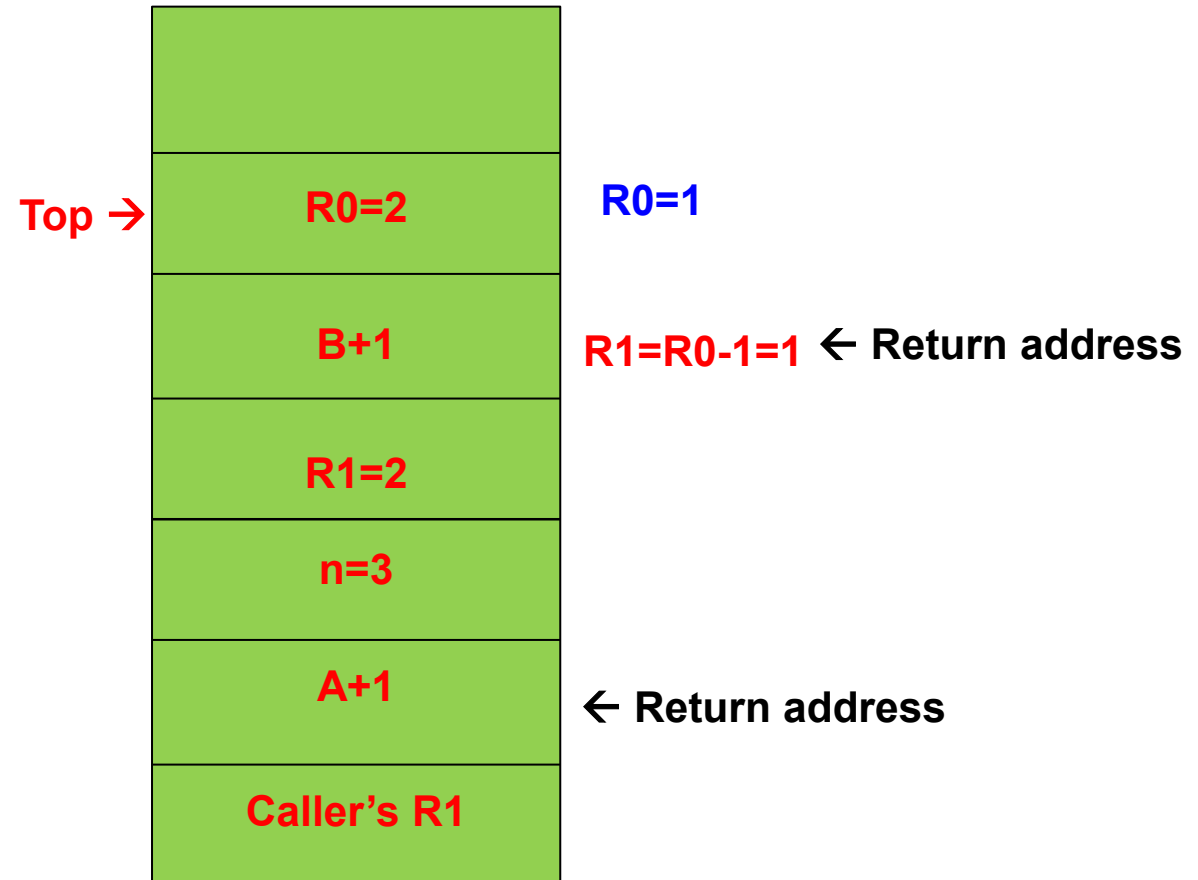
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

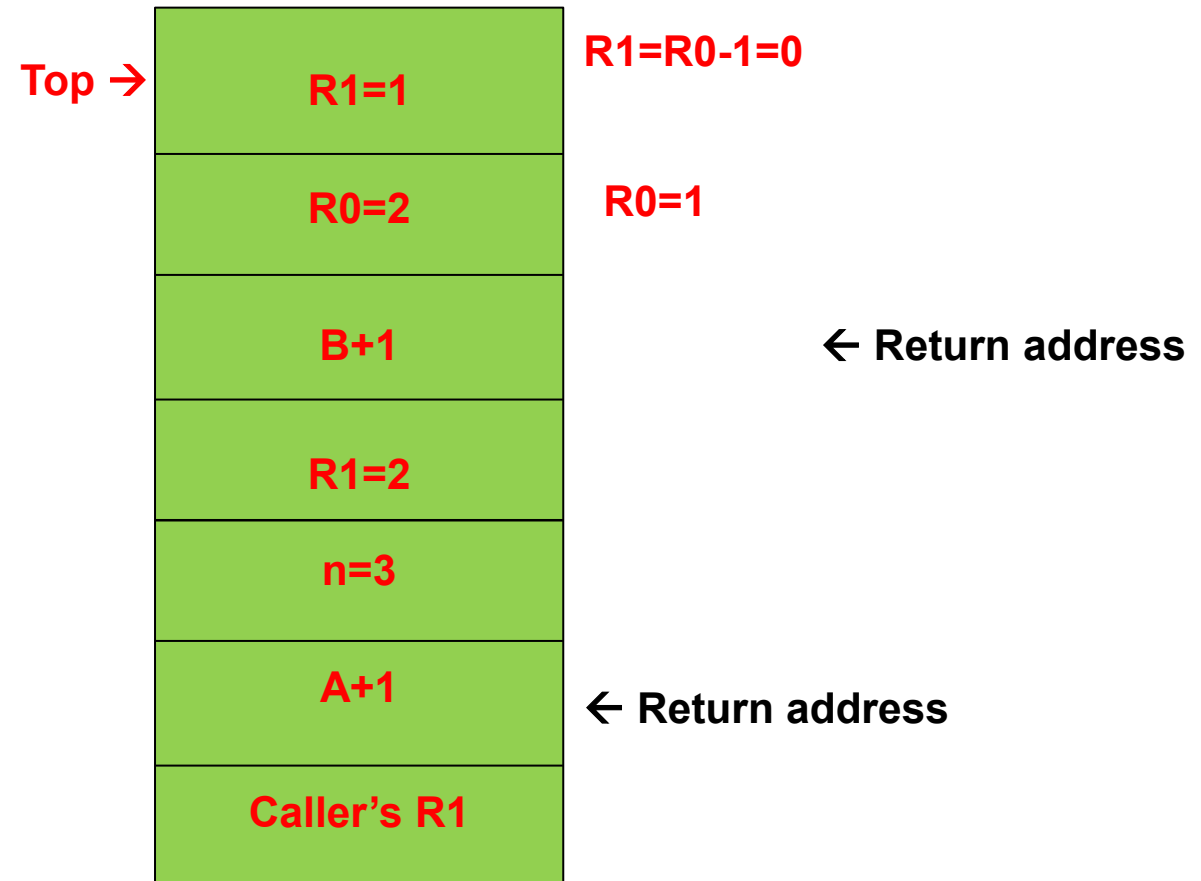
Example: $R0=n=3$



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■          ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■          ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■          ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■          ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

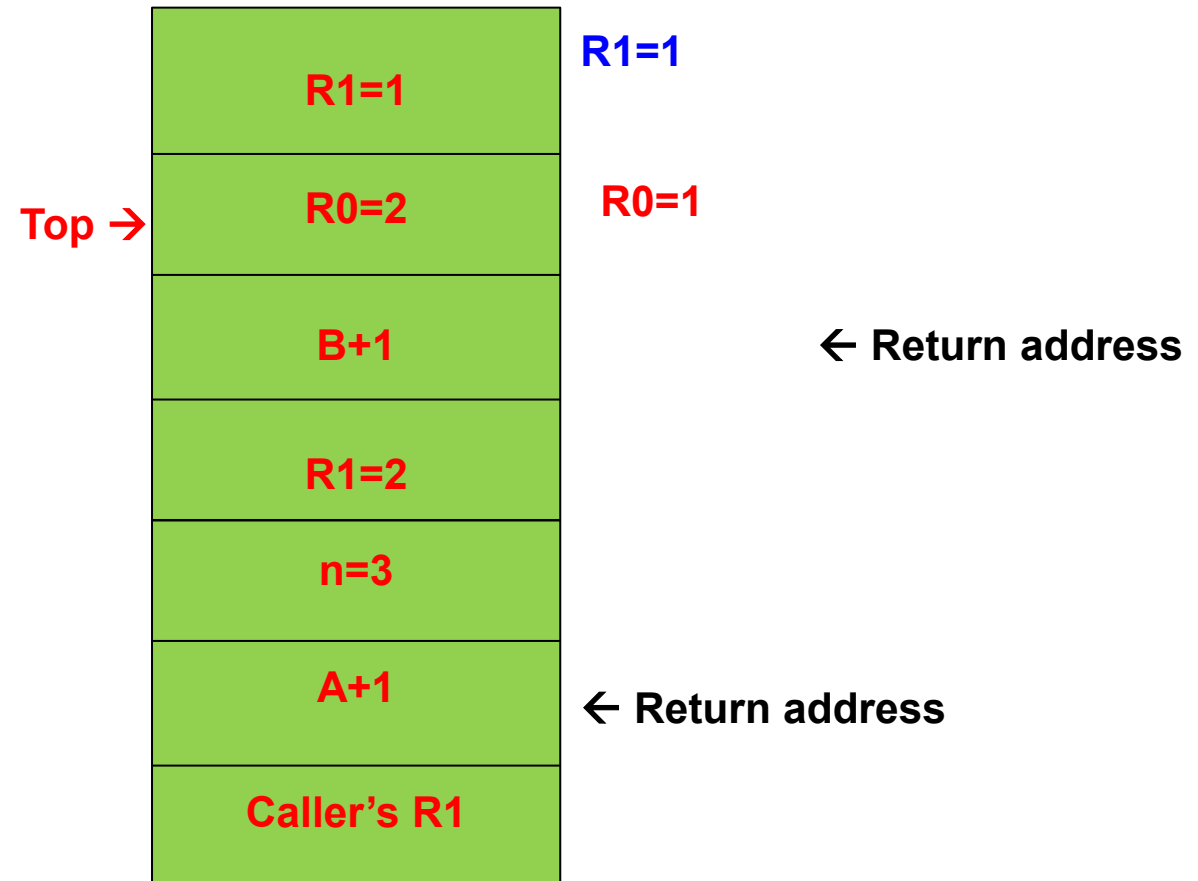
Example: $R0=n=3$



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■          ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■          ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■          ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■          ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

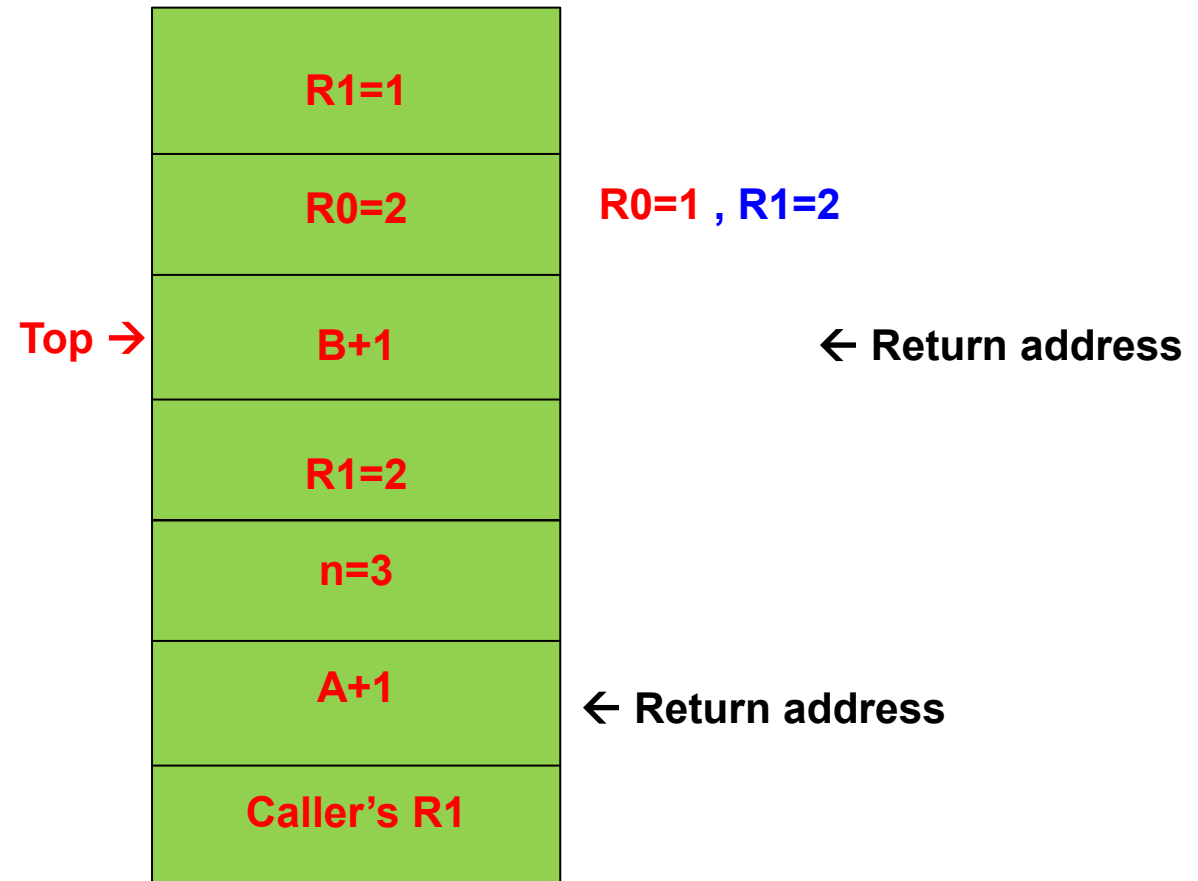
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller' s R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller' s R1 back into R1
■          ADD R6,R6,#1
■          RET
```

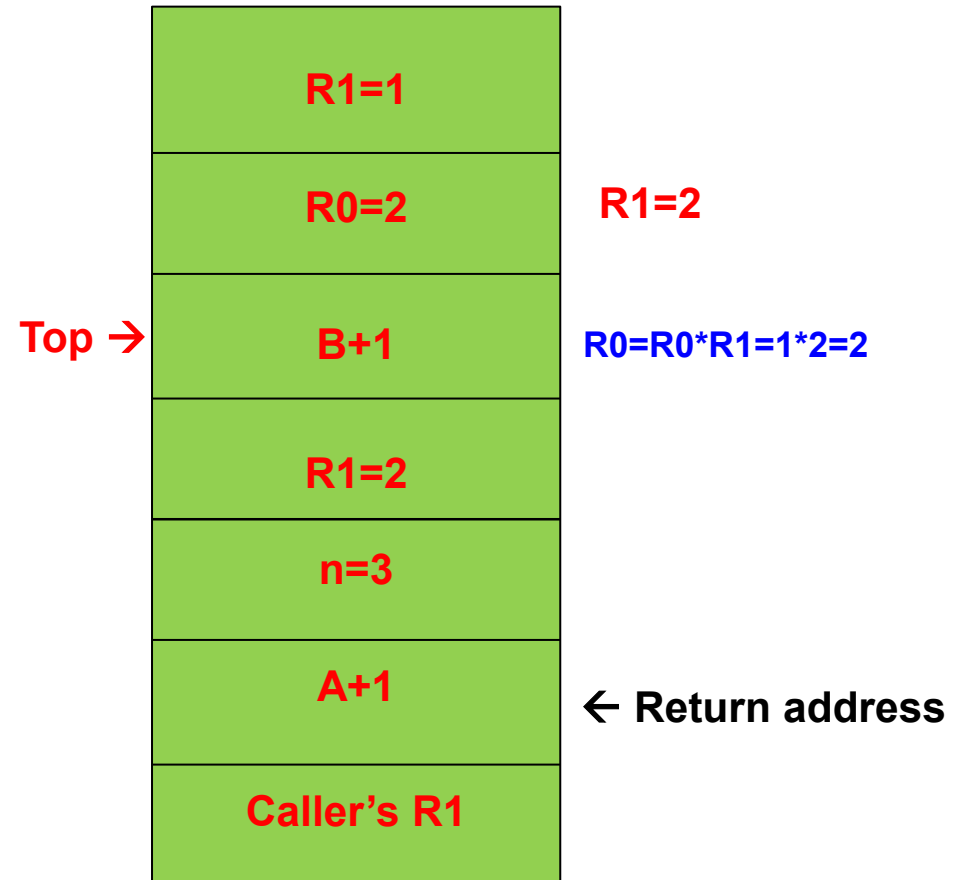
Example: $R0=n=3$



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■          ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■          ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■          ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■          ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

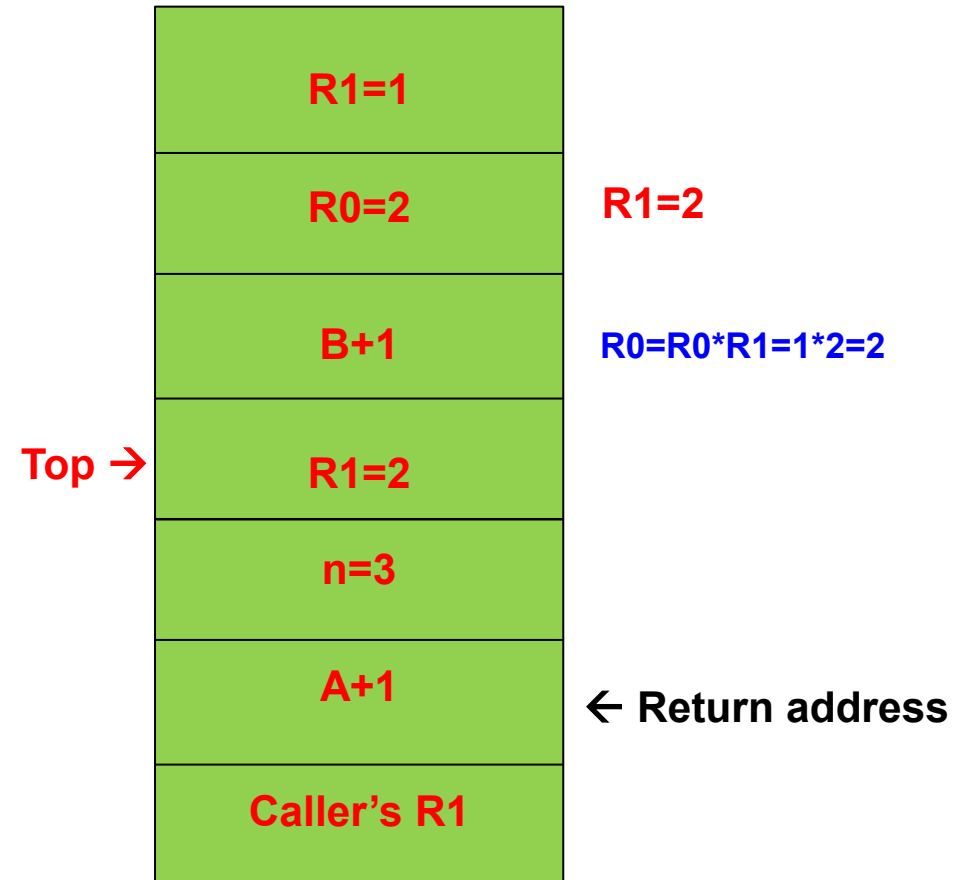
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
■ NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

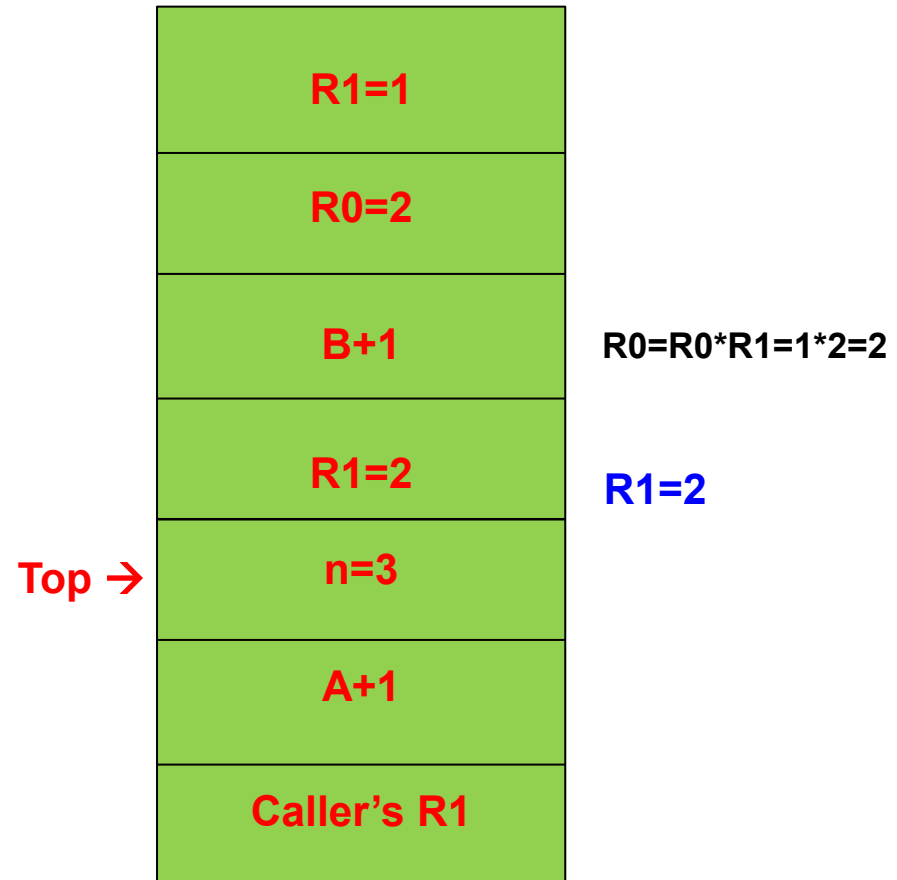
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

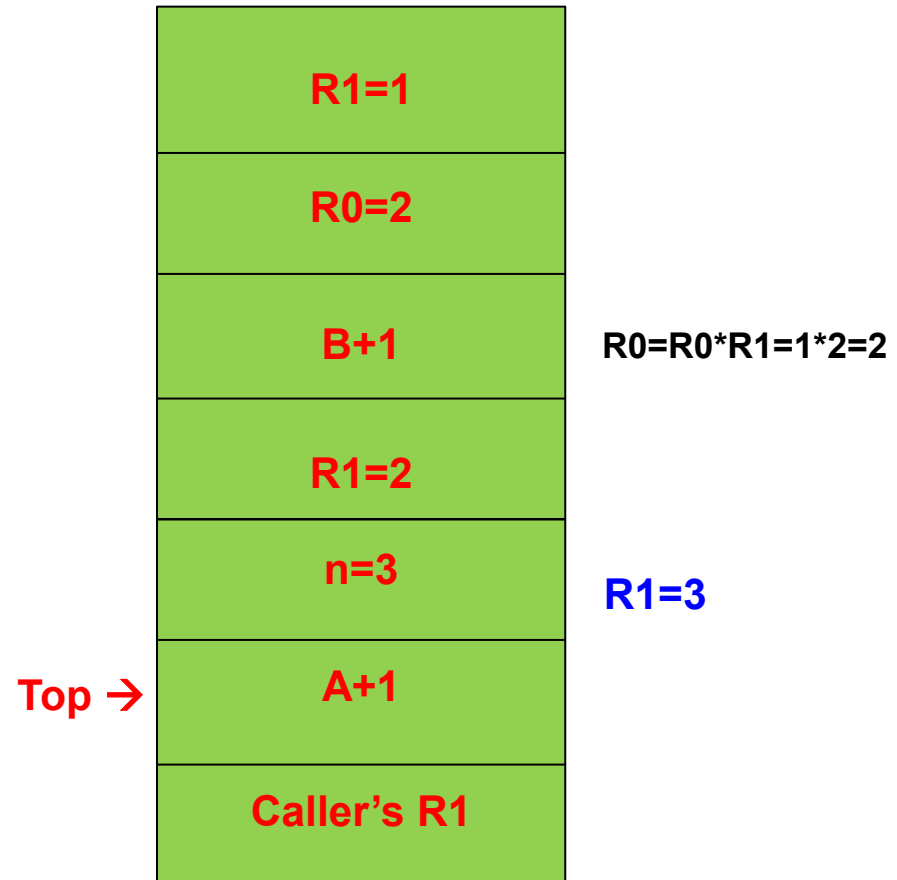
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

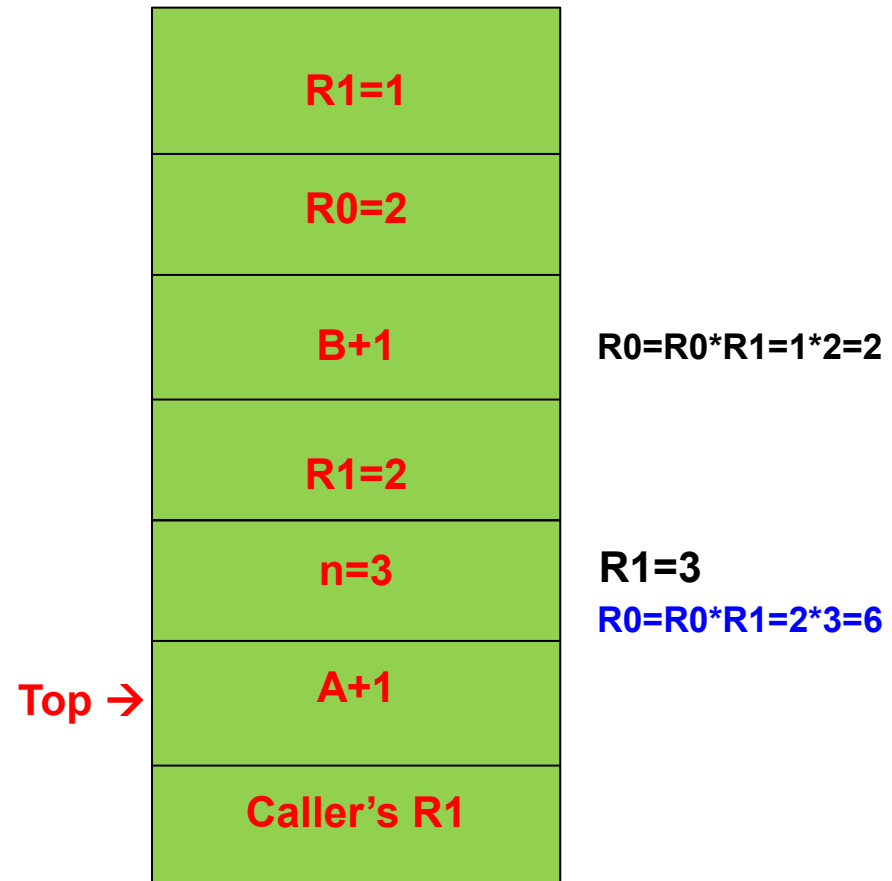
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■          ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■          ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■          ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■          ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

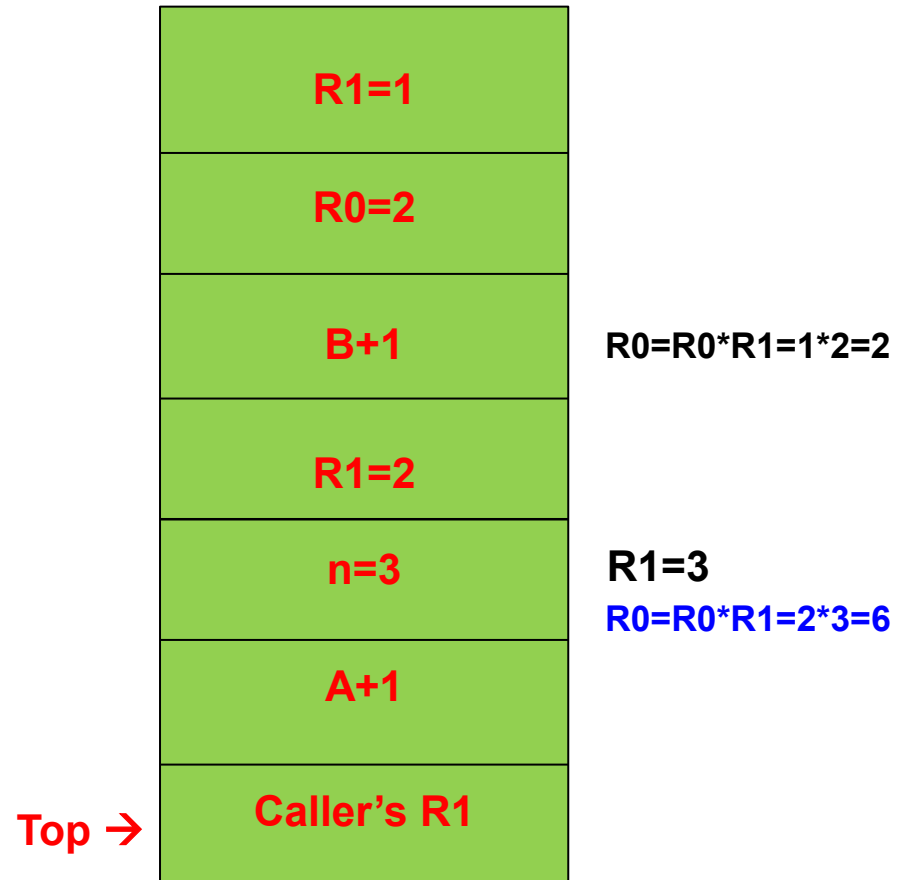
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■ B        JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

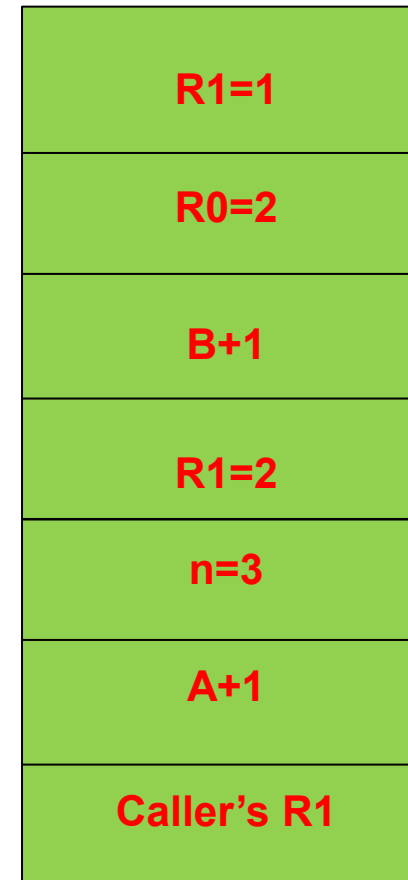
Example: R0=n=3



Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■ ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■ ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■ ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■ B        JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■ ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

Example: R0=n=3



R1=3
R0=R0*R1=2*3=6

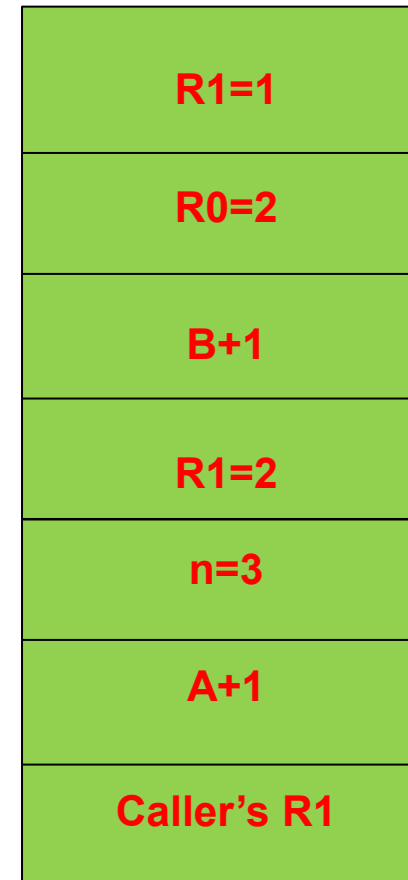
R1=old value

Top →

Recursive subroutine FACT

```
■ FACT    ADD R6,R6,#-1
■          STR R1,R6,#0 ; Push Caller's R1 on the stack, so we can use R1.
■          ;
■          ADD R1,R0,#-1 ; If n=1, we are done since 1! = 1
■          BRz NO_RECURSE
■          ;
■          ADD R6,R6,#-1
■          STR R7,R6,#0 ; Push return linkage onto stack
■          ADD R6,R6,#-1
■          STR R0,R6,#0 ; Push n on the stack
■          ;
■          ADD R0,R0,#-1 ; Form n-1, argument of JSR
■          B    JSR FACT
■          LDR R1,R6,#0 ; Pop n from the stack
■          ADD R6,R6,#1
■          MUL R0,R0,R1 ; form n*(n-1)!
■          ;
■          LDR R7,R6,#0 ; Pop return linkage into R7
■          ADD R6,R6,#1
NO_RECURSE LDR R1,R6,#0 ; Pop caller's R1 back into R1
■          ADD R6,R6,#1
■          RET
```

Example: R0=n=3



R0=R0*R1=2*3=6

R1=old value

Top →

Return to calling program

Implementing FACT iteratively (without recursion)

- FACT ST R1,SAVE_R1
- ADD R1,R0,#0 ; R1=R0=n
- ADD R0,R0, #-1 ; R0=n-1
- BRz DONE
- AGAIN MUL R1,R1,R0 ; (n*(n-1))*(n-2)...
- ADD R0,R0, #-1 ; R0 gets next integer for MUL
- BRnp AGAIN
- DONE ADD R0,R1,#0 ; Move n! to R0
- LD R1,SAVE_R1
- RET
- SAVE_R1 .BLKW 1

The Maze: a Good Example

Given a **maze** and a **starting position** within the maze, write a program that **determines whether or not there is a way out of the maze** from your starting position.

A Maze A maze can be any size, n by m. For example, Figure 8.20 illustrates a 6x6 maze.

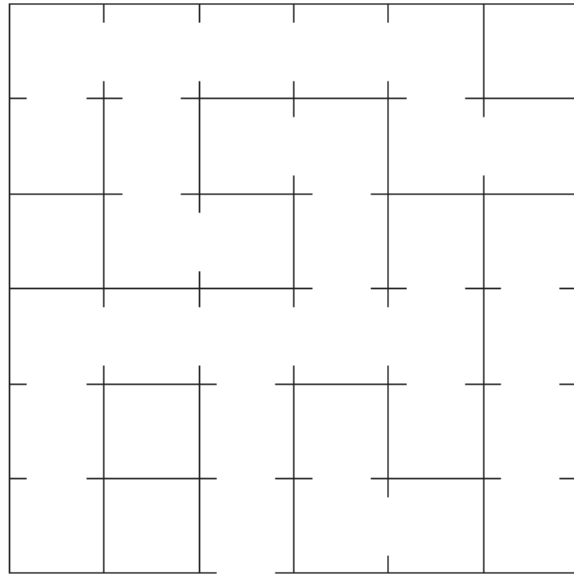
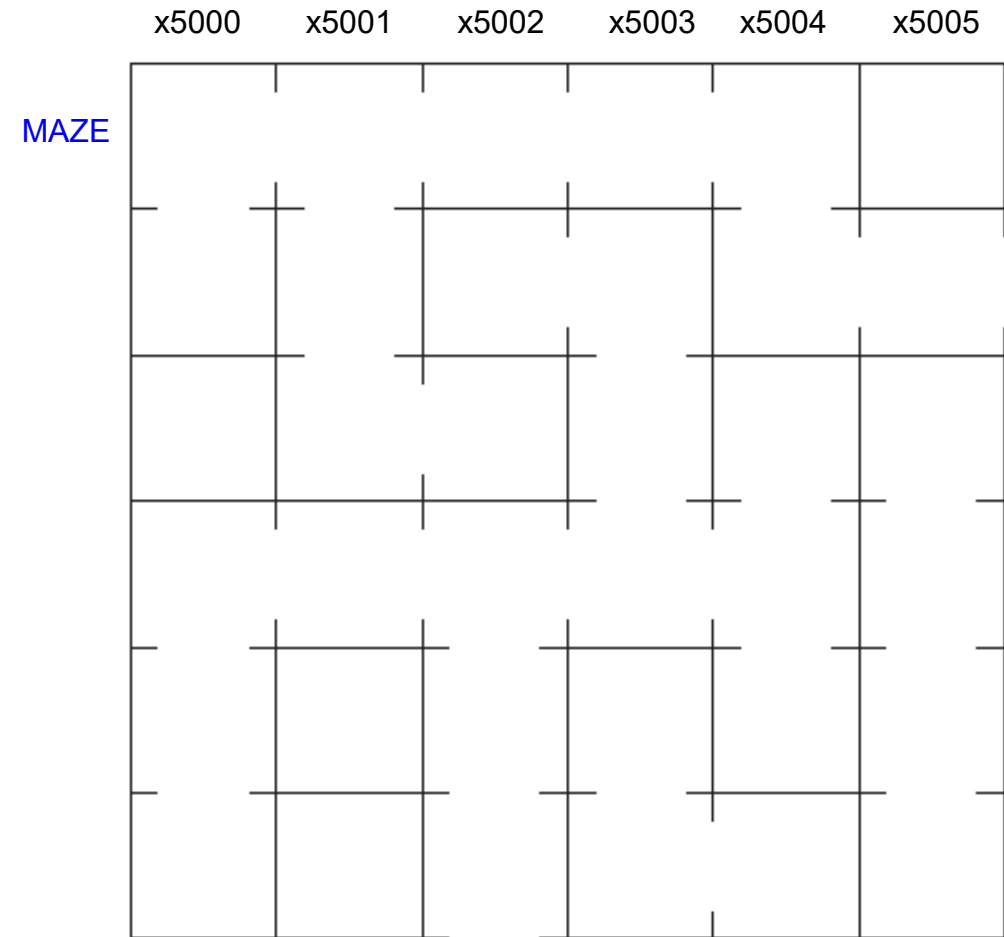


Figure 8.20 Example of a maze.

■ Specification of each cell in the maze

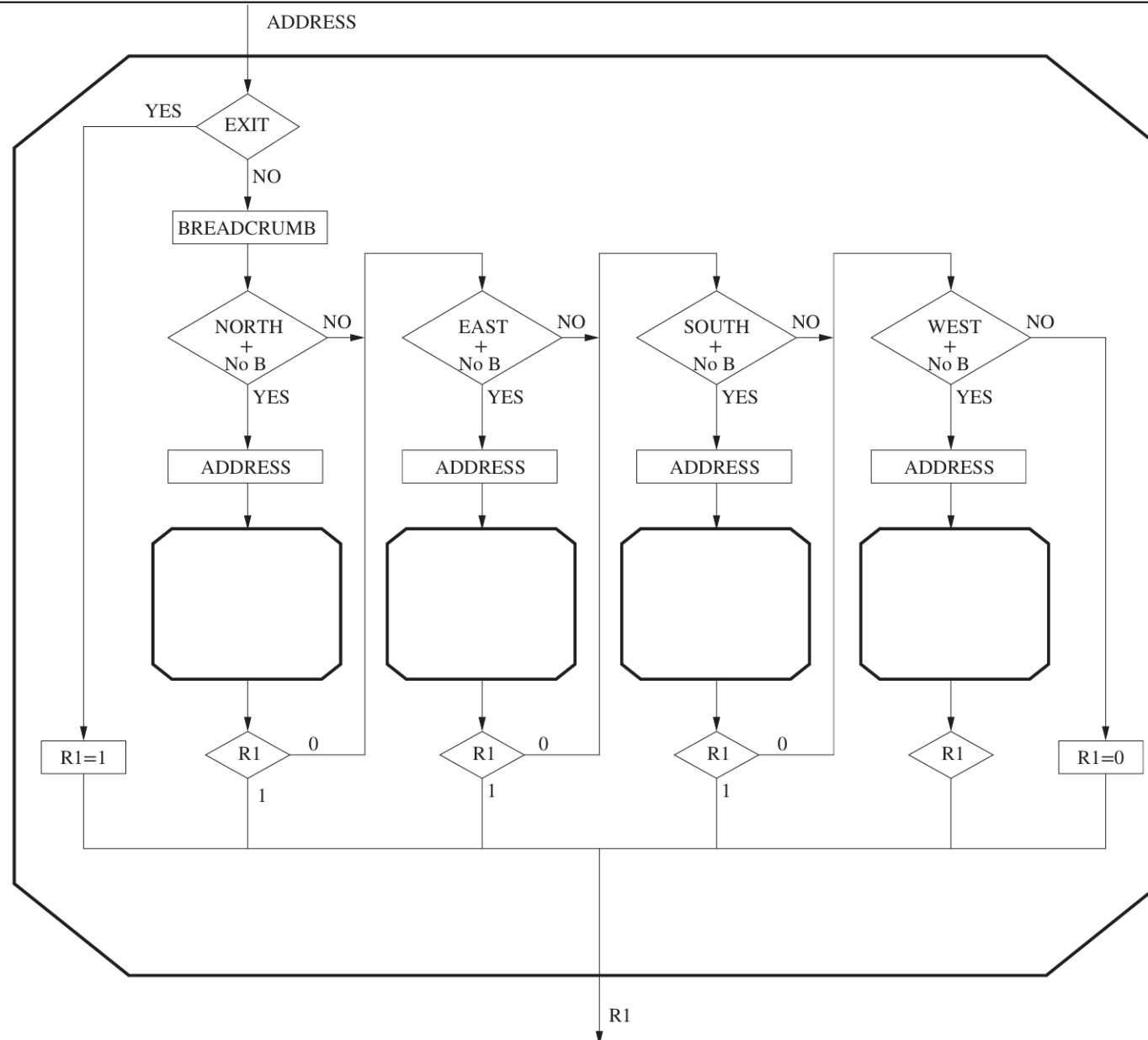
- Bit[4]=1 if there is a door to the outside world; Bit[4]=0 if no door.
- Bit[3]=1 if there is a door to the cell to the north; Bit[3]=0 if no door.
- Bit[2]=1 if there is a door to the cell to the east; Bit[2]=0 if no door.
- Bit[1]=1 if there is a door to the cell to the south; Bit[1]=0 if no door.
- Bit[0]=1 if there is a door to the cell to the west; Bit[0]=0 if no door.

- 00 .ORIG x5000
- 01 MAZE .FILL x0006 ; first row: indices 0 to 5
- 02 .FILL x0007
- 03 .FILL x0005
- 04 .FILL x0005
- 05 .FILL x0003
- 06 .FILL x0000
- 07 ; second row: indices 6 to 11
- 08 .FILL x0008
- 09 .FILL x000A
- 0A .FILL x0004
- 0B .FILL x0003
- 0C .FILL x000C
- 0D .FILL x0015
- 0E ; third row: indices 12 to 17
- 0F .FILL x0000
- 10 .FILL x000C
- 11 .FILL x0001
- 12 .FILL x000A
- 13 .FILL x0002
- 14 .FILL x0002
-
- 2A .END



■ **Figure 8.22** Specification of the maze of Figure 8.20.

Recursive Subroutine to exit the Maze



Description of the Algorithm

- a. From our cell, we ask if we can exit. If yes, we are done. We exit with $R1=1$.
- b. If not, we put a breadcrumb in our cell. Our breadcrumb is bit [15] of the word corresponding to our current cell. We set it to 1.
- c. We ask two questions: Is there a door to the north, and have we never visited the cell to the north before? If the answer to both is yes, we set the address to the cell to the north, and JSR FIND EXIT. We set the address to the cell to the north by simply subtracting 6 from the address of the current cell. Why 6? Because the cells are stored in row major order, and the number of columns in the maze is 6.
- d. If the answer to either question is no, or if going north resulted in failure, we ask: Is there a door to the east, and have we never visited that cell before? If the answer to both is yes, we set the address to the address of the cell to the east (by adding 1 to the address) and JSR FIND EXIT.
- e. If going east does not get us out, we repeat the question for south, and if that does not work, then for west.
- f. If we end up with no door to the west to a cell we have not visited, or if there is a door and we haven't visited, but it results in failure, we are done. We cannot exit the maze from our starting position. We set $R1=0$ and return.

; Recursive subroutine that determines if there is
 ; a path from current cell to the outside world.
 ; **input: R0, current cell address**
 ; **output: R1, YES (1) or NO (0)**

```

      .ORIG x4000
01 FIND_EXIT ; save modified registers into the stack.
02   ADD R6, R6, #-1
03   STR R2, R6, #0 ; R2 holds the cell data of the caller
04   ADD R6, R6, #-1
05   STR R3, R6, #0 ; R3 holds the cell address of the caller
06   ADD R6, R6, #-1
07   STR R7, R6, #0 ; R7 holds the PC of the caller
08
09   ; Move cell address to R3, since we need to use R0
0A   ; as the input to recursive subroutine calls.
0B   ADD R3, R0, #0
0C
0D   ; If the exit is in this cell, return YES
0E   LDR R2, R0, #0 ; R2 now holds the current cell data
0F   LD R7, EXIT_MASK
10   AND R7, R2, R7
11   BRnp DONE_YES
12
13   ; Put breadcrumb in the current cell.
14   LD R7, BREADCRUMB
15   ADD R2, R2, R7
16   STR R2, R0, #0
17

```

```

18 ; check the north cell for a path to exit
19 CHECK_NORTH LD R7, NORTH_MASK
1A   AND R7, R2, R7
1B   BRz CHECK_EAST ; If north is blocked, check east
1C   LDR R7, R3, #-6
1D   BRn CHECK_EAST ; If a breadcrumb in the north
                       ; cell, check east. bit[15]=1, negative
1E   ADD R0, R3, #-6
1F   JSR FIND_EXIT ; Recursively check the north cell
20   ADD R1, R1, #0
21   BRp DONE_YES ; If a path from north cell found,
22                       ; return YES
23 ; check the north cell for a path to exit
24 CHECK_EAST LD R7, EAST_MASK
25   AND R7, R2, R7
26   BRz CHECK_SOUTH; If the way to east is
                       ; blocked, check south
27   LDR R7, R3, #1
28   BRn CHECK_SOUTH ; If a breadcrumb in the
                       ; east cell, check south
29   ADD R0, R3, #1
2A   JSR FIND_EXIT ; Recursively check the east cell
2B   ADD R1, R1, #0
2C   BRp DONE_YES ; If a path from east cell found,
                       ; return YES,

```

```

2E ; check the south cell for a path to exit
2F CHECK_SOUTH LD R7, SOUTH_MASK
30     AND R7, R2, R7
31     BRz CHECK_WEST ; If the way to south is blocked,
                        ;check west
32     LDR R7, R3, #6
33     BRn CHECK_WEST ; If a breadcrumb in the south
                        ;cell, check west
34     ADD R0, R3, #6
35     JSR FIND_EXIT ; Recursively check the south cell
36     ADD R1, R1, #0
37     BRp DONE_YES ; If a path from south cell found,
                        ;return YES
38
39 ; check the west cell for a path to exit
3A CHECK_WEST LD R7, WEST_MASK
3B     AND R7, R2, R7
3C     BRz DONE_NO ; If the way to west is blocked,
                        ; return NO
3D     LDR R7, R3, #-1
3E     BRn DONE_NO ; If a breadcrumb in the west cell,
                        ; return NO
3F     ADD R0, R3, #-1
40     JSR FIND_EXIT ; Recursively check the west cell
41     ADD R1, R1, #0
42     BRp DONE_YES ; If a path from west cell found,
                        ; return YES
43
44 DONE_NO AND R1, R1, #0
45     BR RESTORE

```

```

46
47 DONE_YES AND R1, R1, #0
48     ADD R1, R1, #1
49
4A RESTORE ADD R0, R3, #0 ; restore R0 from R3
;
4B ; restore the rest of the modified registers
;from the stack.
4C     LDR R7, R6, #0
4D     ADD R6, R6, #1
4E     LDR R3, R6, #0
4F     ADD R6, R6, #1
50     LDR R2, R6, #0
51     ADD R6, R6, #1
52     RET
53
54 BREADCRUMB .FILL x8000
55 EXIT_MASK .FILL x0010
56 NORTH_MASK .FILL x0008
57 EAST_MASK .FILL x0004
58 SOUTH_MASK .FILL x0002
59 WEST_MASK .FILL x0001
5A     .END

```

2.The Queue

- **Definition of Queue**
- **Basic Operations**
- **Wrap-Around**
- **Definition of full and empty queue**
- **Underflow/Overflow**

2.1 The Definition of Queue

■ Queue

- A data structure with the property of “First in First out”;
- **Front** pointer for removing elements from the front of the queue; **Rear** pointer for inserting into the rear of the queue.
- **Front** points to the location just in front of the first element in the queue; **Rear** points to the location of the last element in the queue.

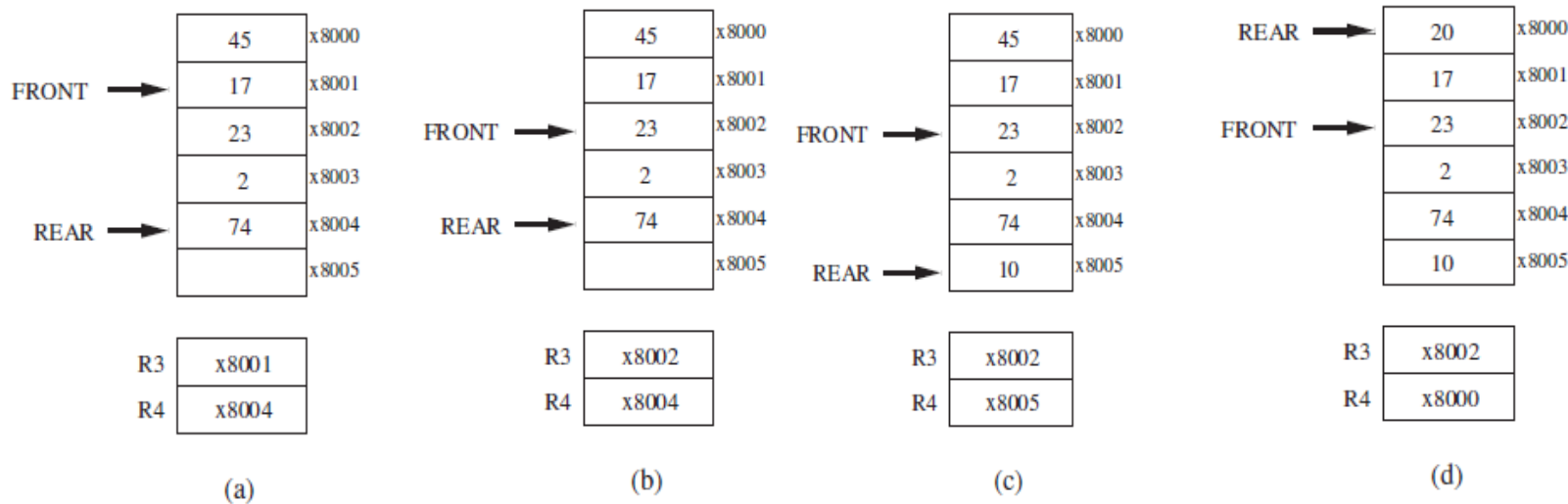


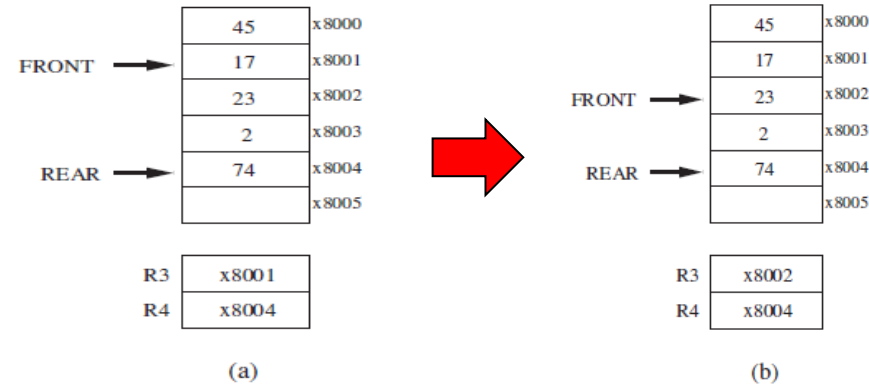
Figure 8.25 A queue allocated to memory locations x8000 to x8005.

2.2 Basic Operations

■ Remove from Front

- FRONT points to the location just in front of the first element in the queue;
 - R3 stores the FRONT pointer
- First incrementing FRONT; then loading the value.

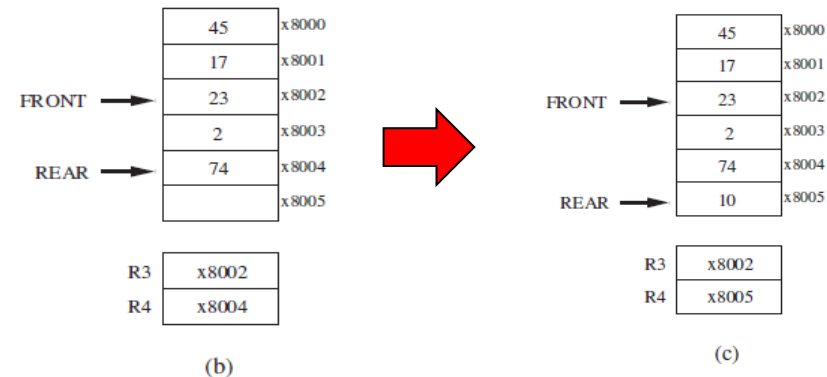
ADD R3,R3,#1
LDR R0,R3,#0



■ Insert at Rear

- First incrementing REAR; then storing the value.
 - R4 stores the REAR pointer

ADD R4,R4,#1
STR R0,R4,#0



2.3 Wrap-Around

■ Remove

;R3 stores FRONT pointer

LD R2, LAST

ADD R2,R3,R2

BRnp SKIP_1

LD R3,FIRST

BR SKIP_2

SKIP_1 ADD R3,R3,#1

SKIP_2 LDR R0,R3,#0

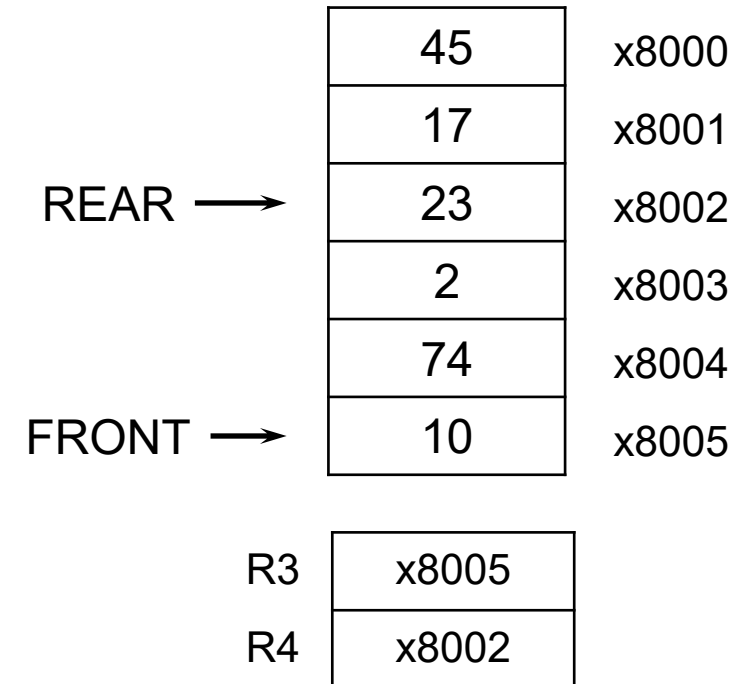
; R0 gets the front of the queue

RET

LAST .FILL x7FFB

; LAST contains the negative of 8005

FIRST .FILL x8000



2.3 Wrap-Around

■ Insert

;R4 stores REAR pointer

LD R2, LAST

ADD R2,R4,R2

BRnp SKIP_1

LD R4,FIRST

BR SKIP_2

SKIP_1 ADD R4,R4,#1

SKIP_2 STR R0,R4,#0

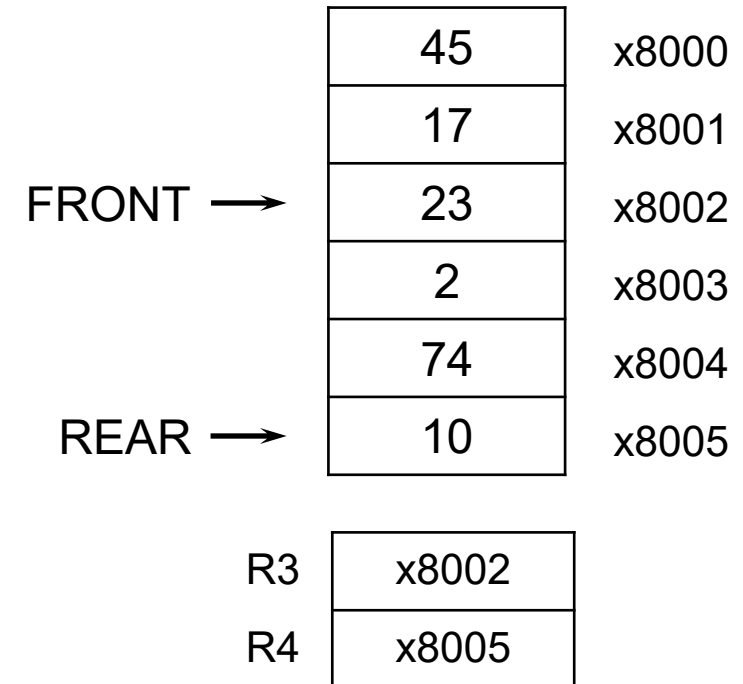
; R0 gets the front of the queue

RET

LAST .FILL 7FFB

; LAST contains the negative of
8005

FIRST .FILL x8000



2.4 Full and Empty Queue

- The queue are allowed to store only $n-1$ elements for a queue with n locations.
- Full
 - $\text{FRONT} = \text{REAR} + 1 \bmod n$
- Empty
 - $\text{FRONT} = \text{REAR}$

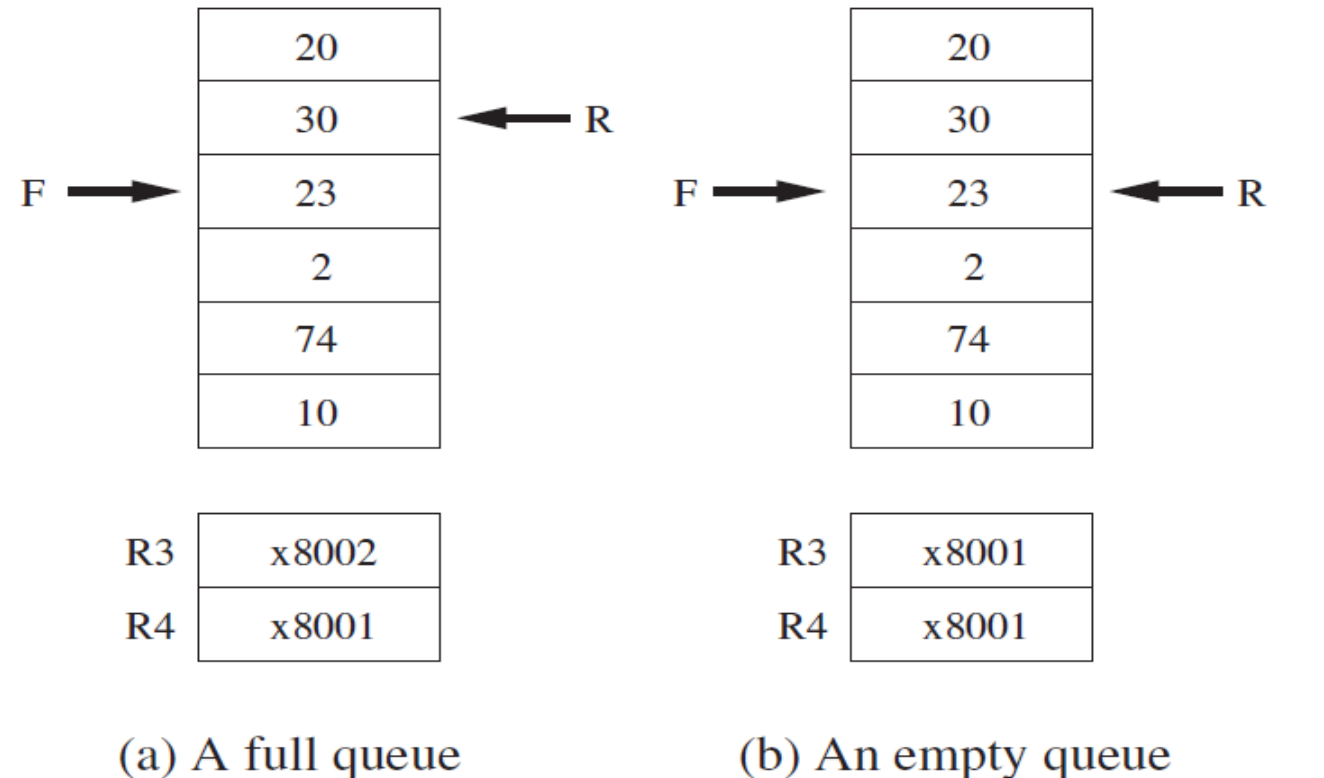


Figure 8.26 A full queue and an empty queue.

2.5 Tests for Underflow, Overflow

R5=1: underflow
R5=0: normal

R3: FRONT
R4: REAR

■ Test for underflow

AND R5,R5,#0

; Initialize R5 to 0

NOT R2,R3

ADD R2,R2,#1

; R2 contains negative of R3

ADD R2,R2,R4

BRz UNDERFLOW

.....

; code to remove the front of the
;queue and return success.

RET

UNDERFLOW ADD R5,R5,#1

RET

2.6 The Complete Story

```
■ 00 ;Input: R0 for item to be inserted, R3 is FRONT, R4 is REAR
■ 01 ;Output: R0 for item to be removed
■ 02 ;
■ 03 INSERT ST R1,SaveR1 ; Save register we need
■ 04      AND R5,R5,#0 ; Set R5 to success code
■ 05 ; Initialization complete
■ 06      LD R1,NEG_LAST
■ 07      ADD R1,R1,R4 ; R1 = REAR MINUS x8005
■ 08      BRnp SKIP1 ; SKIP WRAP AROUND
■ 09      LD R4,FIRST ; WRAP AROUND, R4=x8000
■ 0A      BR SKIP2
■ 0B SKIP1 ADD R4,R4,#1 ; NO WRAP AROUND, R4=R4+1
■ 0C SKIP2 NOT R1,R4
■ 0D      ADD R1,R1,#1 ; R1 = NEG REAR
■ 0E      ADD R1,R1,R3 ; R1 = FRONT-REAR
■ 0F      BRz FULL
■ 10      STR R0,R4,#0 ; DO THE INSERT
■ 11      BR DONE
■ 12 FULL LD R1,NEG_FIRST
■ 13      ADD R1,R1,R4 ; R1 = REAR MINUS x8000
■ 14      BRnp SKIP3
■ 15      LD R4,LAST ; UNDO WRAP AROUND, REAR=x8005
■ 16      BR SKIP4
■ 17 SKIP3 ADD R4,R4,#-1 ; NO WRAP AROUND, R4=R4-1
■ 18 SKIP4  ADD R5,R5,#1 ; R5=FAILURE
■ 19      BR DONE
■ 1A ;
■ 1B REMOVE ST R1,SaveR1 ; Save register we need
■ 1C      AND R5,R5,#0 ; Set R5 to success code
■ 1D ; Initialization complete
■ 1E      NOT R1,R4
■ 1F      ADD R1,R1,#1 ; R1 = NEG REAR
■ 20      ADD R1,R1,R3 ; R1 = FRONT-REAR
■ 21      BRz EMPTY
■ 22      LD R1, NEG_LAST
■ 23      ADD R1,R1,R3 ; R1 = FRONT MINUS x8005
■ 24      BRnp SKIP5
■ 25      LD R3, FIRST ; R3=x8000
■ 26      BR SKIP6
■ 27      SKIP5 ADD R3,R3,#1 ; R3=R3+1
■ 28      SKIP6 LDR R0,R3,#0 ; DO THE REMOVE
■ 29      BR DONE
■ 2A      EMPTY ADD R5,R5,#1 ; R5=FAILURE
■ 2B      DONE LD R1,SaveR1 ; Restore register
■ 2C      RET
■ 2D FIRST      .FILL x8000
■ 2E NEG_FIRST .FILL x8000
■ 2F LAST       .FILL x8005
■ 30 NEG_LAST  .FILL x7FFB
■ 31 SaveR1    .BLKW 1
```

3. Character Strings

- One-dimensional array of ASCII codes often followed by x0000 (null character).
- Each location stores an ASCII codes of a character.

x5000	x0042
	x0069
	x006C
	x006C
	x0020
	x004C
	x0069
	x006E
	x0076
	x0069
	x006C
	x006C
	x0000

Figure 8.28 Character string representing the name “Bill Linvill.”

Example 1: Personnel Record

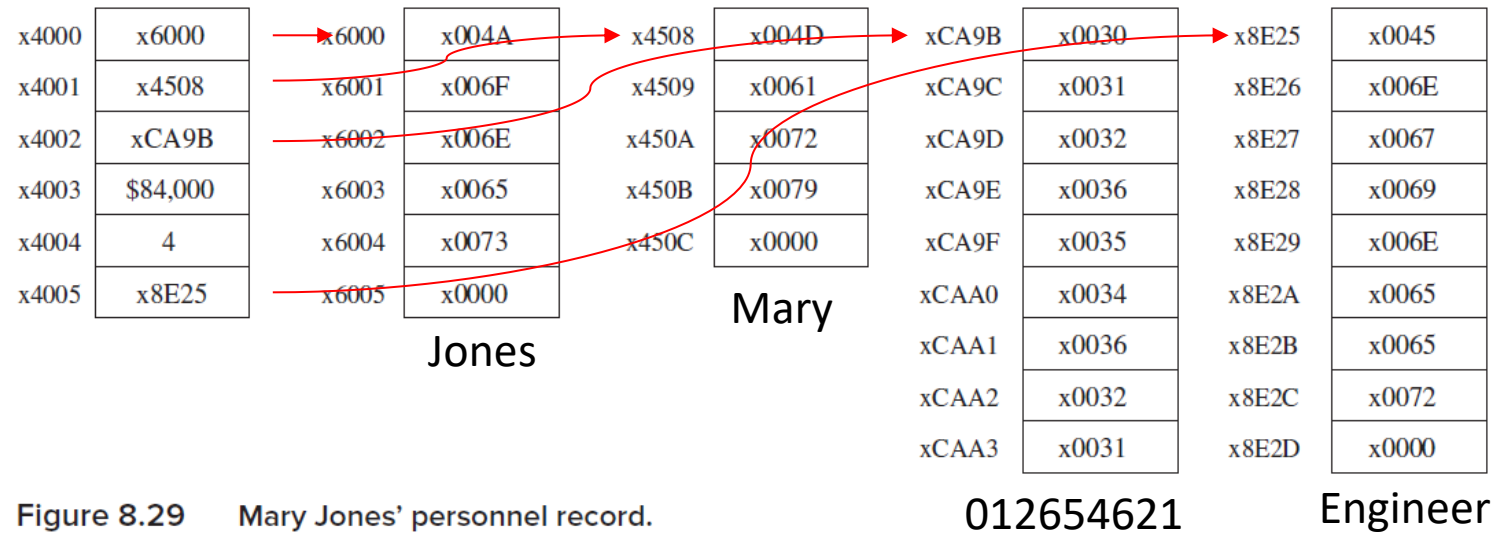


Figure 8.29 Mary Jones' personnel record.

■ 6 words of sequential memory starting at x4000:

- 1. The first word contains the starting address of a character string containing the person's last name. The pointer in location x4000 is the address x6000. The six-word character string, starting at x6000, contains the ASCII code for "Jones," terminated with the null character.
- 2. The second word, at x4001, contains a pointer to the character string of the person's first name, in this case "Mary," starting at location x4508.
- 3. The third word, at x4002, contains a pointer (xCA9B) to her nine-digit social security number, the unique identifier for all persons working in the United States.
- 4. The fourth word, at x4003, contains her salary (in thousands of dollars).
- 5. The fifth word contains how long she has worked for the company.
- 6. The sixth word is a pointer (x8E25) to the character string identifying her job title, "Engineer."

Subroutine to compare two character strings

```
STRCMP ST R0,SaveR0
        ST R1,SaveR1
        ST R2,SaveR2
        ST R3,SaveR3
;R0 points to 1st string, R1 points to 2nd string
        AND R5,R5,#0 ; R5 <-- Match;
NEXTCH LDR R2,R0,#0
; R2 contains character from 1st string
        LDR R3,R1,#0
; R3 contains character from 2nd string
        BRnp COMPARE
; String is not done, continue comparing
        ADD R2,R2,#0
        BRz DONE
; If both strings done, match found
COMPARE NOT R2,R2
        ADD R2,R2,#1
; R2 contains negative of character
        ADD R2,R2,R3 ; Compare the 2 characters
```

```
BRnp FAIL ; Not equal, no match
ADD R0,R0,#1
ADD R1,R1,#1

        BRnzp NEXTCH
; Move on to next pair of characters

FAIL    ADD R5,R5,#1 ; R5 <-- No match
;
DONE    LD R0,SaveR0
        LD R1,SaveR1
        LD R2,SaveR2
        LD R3,SaveR3
        RET
SaveR0 .BLKW 1
SaveR1 .BLKW 1
SaveR2 .BLKW 1
SaveR3 .BLKW 1
```

- Given social security number in example 1, how to look for the salary?

Example 2: Character String Containing an “Integer”

- Represent a long integer of any length by character string, ensuring all character are within 0-9.

```
; ; check whether R0 points to the representation of a long integer.
; Input: R0 contains the starting address of the character string
; Output: R5=0, success; R5=1, failure.
TEST_INTEGER ST R1,SaveR1
; Save registers needed by subroutine
    ST R2,SaveR2
    ST R3,SaveR3
    ST R4,SaveR4
;
    AND R5,R5,#0 ; Initialize success code to R5=0, success
    LD R2,ASCII_0 ; R2=xFFD0, the negative of ASCII code x30
    LD R3,ASCII_9 ; R3=xFFC7, the negative of ASCII code x39
;
NEXT_CHAR LDR R1,R0,#0 ; Load next character
    BRz SUCCESS ; if current character is null
    ADD R4,R1,R2
    BRn BAD ; R1 is less than x30, not a decimal digit
    ADD R4,R1,R3
    BRp BAD ; R1 is greater than x39, not a decimal digit
    ADD R0,R0,#1 ; Character good! Prepare for next character
    BR NEXT_CHAR
```

```
BAD      ADD R5,R5,#1 ; R5 contains failure code
SUCCESS LD R4,SaveR4 ; Restore registers
          LD R3,SaveR3
          LD R2,SaveR2
          LD R1,SaveR1
          RET
ASCII_0 .FILL xFFD0
ASCII_9 .FILL xFFC7
SaveR1  .BLKW 1
SaveR2  .BLKW 1
SaveR3  .BLKW 1
SaveR4  .BLKW 1
```

x0037
x0039
x0032
x0034
x0035
x0000

Figure 8.31 A character string representing the integer 79,245, with one ASCII code per decimal digit.